Int. J. Man–Machine Studies (1979) **11**, 585–620

A microcomputer-based speech synthesis-by-rule system

I. H. WITTEN AND J. ABBESS

Man–Machine Systems Laboratory, Department of Electrical Engineering Science, University of Essex, U.K.

(*Received 9 November 1978*)

This paper describes a system for generating speech by rule from a phonetic representation, using a resonance analogue speech synthesizer driven by parameters which are computed in real time by a microcomputer. Input to it, in the form of a phonetic transcription with additional markers to control rhythm and intonation, can come from a terminal attached to the microcomputer, or from a host computer linked to it by a serial line.

A novel feature of the implementation is the use of a high-level structured programming language, "C", which is compiled on the host and transmitted to the microcomputer as object code. This allows changes to be made quickly to the segmental and suprasegmental synthesis routines, which are still under development, and combines flexibility with ease of use in man–machine applications requiring speech output. To the host, the microcomputer/synthesizer system is simply a character-oriented low-data-rate output device.

Introduction

This paper describes a speech output peripheral which is implemented as a microcomputer driving a resonance speech synthesizer. The host computer controls the peripheral by sending standard phonetic characters, with rhythmic boundaries, intonation focii, and pitch contours marked. Synthesis proceeds from this representation in the microcomputer, which generates, in real time, control parameters for any of the hardware synthesizers attached to it. Speaking begins a fraction of a second after the host has sent the message, and continuous utterances (with normal inter-sentence pauses) of arbitrary length can be generated with no extra storage requirements.

Applications for such a device abound, and some have been described elsewhere (Witten & Madams, 1977; Witten, 1977). Indeed, a machine which generates speech output by concatenating sounds indicated by a low data-rate character-oriented input has been on the market for some time.† The flexibility afforded by the use of a microcomputer gives our system several advantages:

(a) suprasegmental synthesis can be performed, as well as segmental synthesis;
(b) the host-peripheral transmission follows existing conventions for writing utterances;
(c) the software can be modified easily to incorporate new developments in speech synthesis;
(d) different hardware synthesizers can be accommodated.

† Votrax, Vocal Interface Division, Federal Screw Works, U.S.A.

0020-7373/79/050585+36 $02.00/0 © 1979 Academic Press Inc. (London) Limited

A key feature of the implementation is the use of a high-level structured programming language, "C" (Ritchie, 1973), which is compiled on the host and transmitted to the microcomputer as object code. This retains the advantages of high-level-language programming—flexibility, portability, ease of modification, and use of relatively sophisticated debugging tools—while allowing real-time processing to take place where it belongs, in the peripheral. The speech output device can run stand-alone, with utterances typed on a VDU or teletype terminal; or in conjunction with a host computer as a "black-box" output peripheral; or in a development mode, allowing rapid testing of changes to the speech software. For example, a new synthesizer, Computalker,† built primarily for the computer hobbies market, recently became available to us, and it was incorporated into the system in an hour or so by writing a simple device driver (in C) and re-loading the microcomputer.

In operation, the speech peripheral is attached to a host computer system running a normal time-sharing service (Fig. 1). The speech peripheral is autonomous. However, it

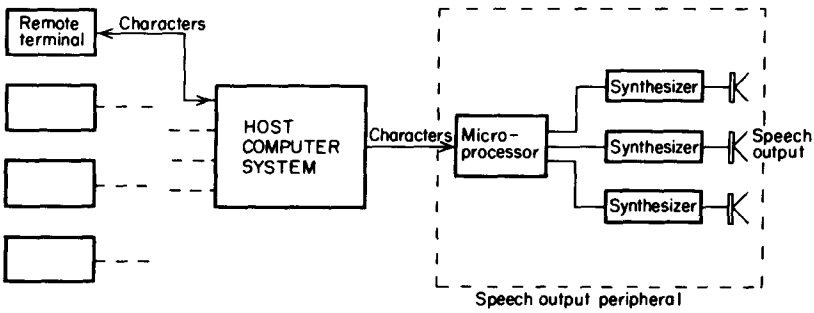


FIG. 1. System configuration.

does not process characters at a constant speed (like normal VDU's or teletypes do); and so the interface handshakes at the hardware level.

We have used an LSI-11 microcomputer to implement the synthesis procedures, in conjunction with a PDP-11/45 host computer. While the coincidence of the instruction sets for these machines has undoubtedly been a help to us when constructing and debugging the system, the high-level language implementation increases its portability—C compilers are becoming available for other microcomputers (Arikan, 1977). Furthermore, when the system is used as a "black-box" peripheral, any computer can serve as the host.

The next two sections describe the operation of the system in more detail. First, the speech synthesis procedure (ISP) is introduced and its linguistic bases are discussed. The input conventions are especially important for a system intended to be used as an ordinary output peripheral by applications programmers. ISP differs from earlier speech-by-rule programs in that rhythm and intonation are computed internally from markers given occasionally in the input, instead of being specified on a per-phoneme basis. In the next section, the computational aspects of ISP are discussed. Considerable overlap of interrupt-initiated activity with background synthesis is necessary in order to achieve the goal of real-time operation in a mocrocomputer, without the delay associated with pre-computing parameters and storing them in a buffer prior to synthesis. Finally, the entire source text of ISP, in C, appears as an Appendix.

† Computalker Consultants, Santa Monica, California, U.S.A.

Synthesis-by-rule procedure

ISP (Integrated Synthesis Program) is a procedure for computing parameters to drive resonance speech synthesizers from a phonemic representation. Such programs are not new. Holmes (1964) developed the first phonetic synthesis-by-rule system, which has since been recoded into machine-independent FORTRAN (Wright, 1976). Holmes' program did not handle the prosodic aspects of rhythm and intonation of speech, which are essential for satisfactory synthesis; instead, phoneme durations and pitch information were explicitly given in the input phonetic string. Mattingly (1966) constructed a procedure for generating this prosodic information from stresses marked in the input, which produced as output an augmented phonetic string suitable for presentation to Holmes' system. Since then, various other synthesis-by-rule systems have been built (Haggard & Mattingly, 1968; Rabiner, 1969; Ainsworth, 1974) along the lines of the original Holmes-Mattingly one, and prosodic phenomena have been extensively investigated (see, for example, Lehiste, 1970; Klatt, 1976).

ISP differs from these earlier efforts in three ways. Firstly, it is relatively synthesizer-independent. The phoneme descriptions in the rule table are given in terms of "logical" parameters (for example, frequencies in Hz), which are translated into physical parameters appropriate to the current synthesizer only just before the parameters are passed to the synthesizer. ISP currently drives both an analogue series formant synthesizer and a digital parallel formant synthesizer.

Secondly, ISP uses a uniform internal data structure which facilitates the passing of information from the suprasegmental analysis down to the segmental level. The suprasegmental tasks of duration and pitch assignment must take into account the whole of each rhythmic unit (the foot (Abercrombie, 1965, p. 28) is used as the basic unit of rhythm) and the whole of each intonation unit (we use Halliday's (1967) tone groups). However, unless one is prepared to buffer synthesizer parameters for the entire utterance before starting to speak it—and this will certainly introduce a long pause before it is said, and require potentially vast buffer storage requirements—the segmental synthesis must proceed in real time. ISP allows information about, for example, syllable boundaries, to be passed easily from the suprasegmental level to the segmental level. It is this integration of the levels that gives ISP its name.

Thirdly, ISP is designed to operate in real time in a microcomputer, which appears to the host computer to be a phonemically-driven output device. Considerable attention is paid to the overlapping of real-time interrupt-initiated synthesizer control with the segmental synthesis-by-rule. In addition, since tone groups, which are the basic macro-units of synthesis, often follow directly after one another, the segmental synthesis is itself overlapped with suprasegmental analysis of the *next* tone group. This allows arbitrarily long utterances, comprising many tone groups, to be synthesized without breaks, with quite modest internal buffering requirements.

INPUT CONVENTIONS

Sentences to be input to ISP are written in the form

```
1  dh i s  i z  dh uh  /h a uu s  dh uh t  /*d zh aa k b i l t;
```

or, equivalently,

```
1  dhis  iz  dhuh  /hauus  dhuht  /*dzhaak  bilt.
```

The standard International Phonetic Association symbols are coded into one or two letters to cope with the limitations of normal computer input devices. Slashes mark the *foot boundaries* (which correspond to stressed syllables), and an asterisk identifies the point of major stress. Utterances are divided into *tone groups*, and the shape of the intonation contour is specified by a code ("1", in this case) at the start of each one.

Tone groups are terminated by normal punctuation marks: ".", "!", ",", ";". The first two represent final pauses, while the last are non-final in that synthesis continues past them without a break. These punctuation marks, together with tone-group identifiers and associated information, a foot-boundary symbol (we use "/"), a stressed foot-boundary symbol (we use "/*"), a word-boundary symbol (" "), a syllable-boundary symbol ("'"), the phoneme names and associated data, a duration marker (for explicitly specifying phoneme durations, if desired), a pitch marker (for explicitly specifying pitches, if desired), and various miscellaneous parameters, are all contained in a rule table which is read by ISP on loading.

After reading a tone group, ISP passes it to a rhythm assignment procedure, and then to a pitch assignment routine. Only when this prosodic analysis is complete does segmental synthesis begin.

RHYTHM

The rhythm assignment has been described in detail elsewhere (Witten & Smith, 1977). It is founded upon the theory of isochrony of feet (Abercrombie, 1965, p. 28) which states that, other things being equal, each foot of the utterance occupies the same duration. Basically, the utterance is segmented into syllables, and the beginning of each is noted. Syllable boundaries may optionally be marked in the input string, in which case they are respected. The types of the syllables ("strong", "weak", "enclitic", "proclitic") are identified. The pattern of syllable types in each foot is used to access a look-up table which determines how the foot duration is to be divided between the syllables. Then the phoneme clusters which make up each syllable (e.g. ⟨h⟩⟨aa uu⟩⟨s⟩; ⟨b⟩⟨i⟩⟨l t⟩; ⟨s p l⟩ ⟨i⟩⟨t⟩) are examined and classified into types, the pattern of types again dictating how the syllable duration is to be divided among the clusters via a look-up table. Finally, the cluster durations are allocated between the individual phonemes in a similar manner. The phoneme durations are not allowed to exceed a maximum which is given in the rule table, and this maximum depends upon whether the parent foot is utterance-final and tonic-stressed in a way which allows such feet to be considerably longer than the others, as is observed in human speech (Lehiste, 1973).

Recent work has, however, cast doubt upon the value of such an intricate rhythmic analysis based on feet (Hill *et al.*, 1977*a*,*b*). Nevertheless, it is felt that much of the analysis, such as the division into syllables, will be useful at the segmental synthesis level to incorporate well-known phonetic phenomena (like the accentuated aspiration of stressed-syllable-initial plosives (O'Connor, 1973)).

INTONATION

ISP uses a method of pitch specification which is intended to permit close control over those aspects of an intonation contour that affect the way it is perceived. Ten pitch parameters are defined, each with a simple meaning (Witten, 1978, describes the parameters). A specification of ten values, one for each parameter, constitutes a pitch *contour.* An arbitrary number of pitch contours may be defined in the rule table, each

being given an identifier (we use "1", "1+", "1−", "2", "2−", "3",..., which is Halliday's (1967) notation). Whenever one of these identifiers appears at the beginning of a tone group, its corresponding pitch contour is used. If no such identifier appears, the group is intoned with the contour which appears first in the rule table.

SEGMENTAL SYNTHESIS

The segmental synthesis procedure is based on that described by Hill (1978), with considerable modifications and simplifications. A higher-level, interruptable process makes entries in a linked event-list. Each event has an event time, and the list is kept in time order. Events are of three kinds:

(1) target events, where a logical parameter number, target parameter value, and time-increment are specified;
(2) increment events, where a logical parameter number, constant parameter-increment throughout the event, and time-increment during which the event is in force, are specified;
(3) aspiration events, where the time-increment during which the speech is aspirated is specified.

A low-level interrupt-driven process extracts the events from the list as they become due and makes appropriate modifications to the real-time synthesis data-base.

High-level segmental synthesis is entirely in terms of *logical synthesizer parameters*, which are mapped on to the physical parameters of the synthesizer in question at interrupt level. Logical parameters are:

av amplitude of voicing (linear, between 0 and 63);

as amplitude of sibilance (linear, between 0 and 63);

f1 frequency of formant 1 (Hz);

f2 frequency of formant 2 (Hz);

f3 frequency of formant 3 (Hz);

fh frequency of sibilance formant (Hz);

f0 frequency of fundamental (Hz).

The aspiration/voicing switch which is needed for unvoiced plosives and "h" sounds is not a separate logical parameter, but is controlled solely by the "aspiration event" mentioned above. Throughout such an event, the voicing amplitude av is directed to the aspiration amplitude channel of the synthesizer.

Physical parameters depend upon the synthesizer's organization—in particular whether it is serial or parallel. In the latter case, formant amplitudes are computed from the formant frequencies, just before the phonemes are fed to the synthesizer, using the approximate relationships

$$a_2 = a_1 - 6 * \log_2 (f_2/f_1) \quad \text{dB},$$

$$a_3 = a_2 - 6 * \log_2 (f_3/f_2) \quad \text{dB};$$

where a_1 is the overall amplitude in dB, and a_2 and a_3 are the second and third formant amplitudes.

The basic high-level synthesis process normally inserts, for each phoneme, one target event into the event-list for each logical parameter. The event time is the beginning of the transition into that phoneme, and the event time-increment is the length of the transition. The target parameter values for each phoneme are given in the rule table.

However, a host of exceptions to this basic process occur, and these are triggered by flags stored with each phoneme in the rule table. For example, transitions out of stops have fast amplitude transitions, short sibilance bursts (implemented by increment events), and longer aspirated periods (implemented by aspiration events). If unvoiced phonemes occur directly before and after a voiced stop, the stop gap is unvoiced. If the new phoneme is a "w", "l", or "y", then the formant 3 value of the previous phoneme is retained throughout the glide.

The activity of the low-level synthesis process is dictated by the events in the event-list. At each synthesizer clock tick, any events which have become due are extracted from the list, scaled to suit the synthesizer in question, and written into "target", "increment", and "aspiration" fields of a data-base. Then the new formant values are computed from this data-base, converted from logical to physical parameters, and stored temporarily, to be sent to the synthesizer at the next clock tick.

Computational aspects

In common with other microcomputers, the LSI-11 is quite slow, with a mean instruction execution time of around 8-μs. Since each of the three synthesizers that we have used interrupts every 10 ms, only a thousand or so instructions can be executed between interrupts. However, rather than tailor the software for speed from the start, it was decided to maximize the maintainability and modularity of the program by applying principles of structured programming (Dahl *et al.*, 1972) and tackle any real-time problems if and when they arose.

In the event, little difficulty was encountered in making the software run in real time. It was found necessary to utilize the hardware parameter buffer in the synthesizers to share the load of interrupt handling at critical times over two 10 ms clock periods (as described more fully below); other than this no special tricks were used. In fact, the interrupt period can be reduced to 6 or 7 ms before interrupts are missed, indicating some spare capacity. However, it is not possible to synthesize different utterances in parallel and route them to different synthesizers.

THE SYNTHESIS LIST

The key to the operation of ISP is the *synthesis list.* When an utterance is read from the host computer, the lexical items which comprise it are read and stored in a linked list (see Fig. 2). Subsequent operation of the rhythm and pitch assignment routines serve to augment this list with syllable boundaries, phoneme cluster identifiers, and duration and pitch specifications. Next, the list is passed to the high-level segmental synthesis routine which chains events into the appropriate places in the list and, as it proceeds, removes the no longer useful elements (phoneme names, pitch specifiers, etc.) which originally constituted the synthesis list. Finally, the interrupt-driven speech synthesizer handler removes events from the list as they become due and uses them to update its data-base.

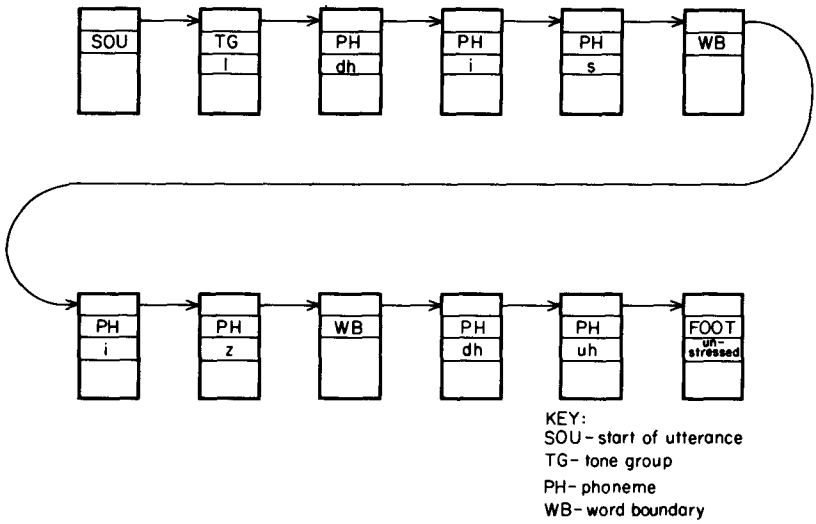


FIG. 2. Start of the synthesis list created for "1 dh i s i z /*d zh aa k s /h aa u s".

By adopting the synthesis list as a uniform data structure for holding utterances in every stage of processing, the problems of storage allocation and garbage collection are minimized. Each list element has a forward pointer and five data words, the first indicating what type of element it is.

Lexical items which may appear in the input are

end of utterance (".", "!");
tone-group identifier ("1", "1+", "1−", . . .);
end of tone-group (",", ";");
foot boundary ("/", "/*");
word boundary (" ");
syllable boundary ("-");
phoneme ("ar", "b", "ng", . . .);
duration specifier for previous phoneme ("dd ⟨number⟩");
pitch specifier ("pp ⟨ number⟩⟨number⟩").

Syllable boundaries and duration and pitch specifiers are optional. If not given in the input they will be computed by ISP. The actual characters which make up lexical items are not fixed but must be given in the rule table—the ones in parentheses above are used in our current table. Additional items which may be inserted by ISP into the synthesis list are

start of utterance;
cluster type (inserted during rhythm assignment).

Further, as described earlier, three types of events—target events, increment events and aspiration events—appear in the list during synthesis. In fact, since the other entries are deleted from the list as synthesis proceeds, it is more correct to say that the synthesis list is *transformed* in the event-list.

INTERRUPT HANDLING

When segmental synthesis commences, the first few phoneme segments of the utterance are processed before the synthesizer is allowed to interrupt the processor. Once interrupts are enabled, segmental synthesis ceases and the processor is free to read the next utterance and perform prosodic analysis on it. Meanwhile, synthesizer interrupts are processed, events are removed from the event-list as they become due, and so the amount of segmental synthesis in hand on the current utterance decreases. When this falls below a low-water mark, the segmental synthesis routine is re-initiated *from interrupt level* (but with interrupts enabled, so that the synthesizer continues to be handled in real-time) to ensure an adequate reserve. When the current utterance has finished, the prosodic analysis of the next utterance should be complete and segmental synthesis of it can begin at once.

The real-time requirements of speech synthesis from a phonetic representation are quite demanding. Between 10 ms interrupts, it is necessary to compute the synthesizer parameters which form the next sample, and this may involve removing events from the event-list. Furthermore, the high-level task of transforming phonemes into events must proceed fast enough to keep pace with the synthesizer. We found it essential to keep the processor in an interruptable state as much as possible, so that should an interrupt not be serviced in time, this could be reported.

The nature of speech is such that there are occasions when several events will become due at the same time—namely, at the beginning of transitions between phoneme segments. (This is exacerbated by our simple-minded segmental synthesis policy in which formant transitions always occur together. Natural speech is more complicated.) It was found that the microcomputer was not fast enough to perform the routine interrupt-handling *and* extract and process several new events (as many as seven, the number of logical synthesizer parameters) in the 10 ms period available. However, it is certain that there will be less activity in the subsequent interrupt period, for phoneme transitions cannot occur on consecutive clock ticks. Furthermore, the synthesizer hardware is such that parameter values requested by one interrupt are not in fact used until the next one occurs, when they are sent from digital buffers to D–A converters. Use is made of this to spread event dequeueing at interrupt level over two 10 ms periods, by allowing an interrupt to remain pending if it occurs during servicing of the previous interrupts. If a further interrupt occurs while one is pending, this is noted as an error.

ERROR REPORTING

After each utterance, any of five error conditions may be returned to the host computer:

interrupt received before the next parameter values were ready;
overdue event encountered in the list;
illegal event type encountered in the list;
event with a negative increment-time encountered;
out-of-range parameter value discovered.

The last condition can occur because, although the formant and fundamental frequencies are specified in Hz in the rule table, the hardware synthesizers cannot necessarily cope with extreme values.

Conclusions

Real-time synthesis-by-rule of speech has been implemented in a microcomputer, using a variety of hardware formant synthesizers. Both segmental and suprasegmental procedures were included. The processor that has been used, an LSI-11, is a 16-bit machine with a relatively powerful instruction set. The system, which is written entirely in a high-level language and designed for maintainability and modularity rather than for speed or size, occupies 16K words of store and runs faster than real time (with 10 ms synthesizer interrupts). Multiple synthesizer channels cannot be handled simultaneously because too much interrupt overhead would be involved.

No doubt the system could be made to handle two independent channels simultaneously, with more buffering of parameters, and more careful coding. However, it is unlikely that many more channels could be accommodated.

It was found useful to incorporate several features which assist debugging and serve as tools to improve the speech quality. For example, the output of the suprasegmental processing can be returned to the host for examination and modification. Additional information may be placed in the input to override the automatic duration and pitch assignment. Parameter tracks computed by the program may be returned to the host, edited there, and sent via the microcomputer for resynthesis.

As for debugging aids, the system monitors its real-time behaviour very closely by keeping the processor in an interruptable state almost all of the time and using a software status word to determine its interruptability. This allows all real-time errors to be detected and reported. Several other internal errors are also caught (as are unexpected processor traps), and returned to the host. A memory-dumping facility is currently being commissioned to return the current contents of the microcomputer's store to the host whenever a fatal error occurs, to be examined with existing post-mortem debugging software.

Many of these features could now be removed, resulting in much more compact code. However, we intend to develop the synthesis system in order to improve the intelligibility and naturalness of the speech, and a rapid edit-debug-modify cycle using intimate connection between the host and the peripheral will greatly facilitate this.

This work is part of a joint project in man–machine communication with the Department of Computer Science, University of Calgary, Canada. The support and encouragement of David Hill, of that Department, is very much appreciated.

References

ABERCROMBIE, D. (1965). *Studies in Phonetics and Linguistics.* Oxford University Press.

AINSWORTH, W. A. (1974). Performance of a speech synthesis system. *International Journal of Man–Machine Studies,* **6**, 493–511.

ARIKAN, R. (1977). *MC Reference Manual.* Computer Systems Laboratory, Queen Mary College, London.

DAHL, O. J., DIJKSTRA, E. W. & HOARE, C. A. R. (1972). *Structured Programming.* London: Academic Press.

HAGGARD, M. P. & MATTINGLY, I. G. (1968). A simple program for synthesizing British English. *IEEE Transactions on Audio and Electroacoustics,* **AU-16**, 95–99.

HALLIDAY, M. A. K. (1967). *Intonation and Grammar in British English.* Paris: Mouton.

HILL, D. R. (1978). A program structure for event-based speech synthesis by rules within a flexible segmental framework. *International Journal of Man–Machine Studies,* **10**, 285–294.

HILL, D. R., JASSEM, W. & WITTEN, I. H. (1977*a*). A statistical approach to the problems of isochronicity. *Proceedings of the International Phonetics Society Conference*, Florida, December.

HILL, D. R., WITTEN, I. H. & JASSEM, W. (1977*b*). Some results from a preliminary study of British English rhythm. Paper presented at the 94th meeting of the Acoustical Society of America, Florida, December.

HOLMES, J. N., MATTINGLY, I. G. & SHEARME, J. N. (1964). Speech synthesis by rule. *Language and Speech*, **7**, 127–143.

KLATT, D. H. (1976). Linguistic uses of segmental duration in English: Acoustic and perceptual evidence. *Journal of the Acoustical Society of America*, **59**, 1208–1221.

LEHISTE, I. (1970). *Suprasegmentals*. MIT Press.

LEHISTE, I. (1973). Rhythmic units and syntactic units in production and perception. *Journal of the Acoustical Society of America*, **54**, 1228–1234.

MATTINGLY, I. G. (1966). Synthesis by rule of prosodic features. *Language and Speech*, **9**, 1–13.

O'CONNOR, J. D. (1973). *Phonetics*. London: Penguin.

RABINER, L. R. (1969). A model for synthesizing speech by rule. *IEEE Transactions on Audio and Electroacoustics*, **AU-17**, 7–13.

RITCHIE, D. M. (1973). *C Reference Manual*. Murray Hill, N.J.: Bell Telephone Laboratories.

WITTEN, I. H. (1977). Software for speech output. *Proceedings of the IFIP Congress 77*, Toronto, August, pp. 297–391.

WITTEN, I. H. (1978). A flexible scheme for assigning timing and pitch to synthetic speech. *Language and Speech*, **20**, 240–260.

WITTEN, I. H. & MADAMS, P. H. C. (1977). The Telephone Enquiry Service: a man–machine system using synthetic speech. *International Journal of Man-Machine Studies*, **9**, 449–464.

WITTEN, I. H. & SMITH, A. (1977). Synthesizing British English rhythm—a structured approach. *Proceedings of the Fifth Canadian Man–Computer Communication Conference*, Calgary, May, pp. 175–185.

WRIGHT, R. D. (1976). A system for implementing rule synthesis of speech. *Proceedings of the Institute of Acoustics Autumn Conference*, Edinburgh, September, pp. 2-20-1–2-20-4.

Appendix: ISP listing

isp.h

This first part defines the data structures and parameters.

```
#
/*
 *                      ISP global definitions
 *
 *
 * 1  DATA STRUCTURES
 *
 * 1.1  The rule table
 *
 *      This comprises four parts:  the lexical character array
 *      (lex), the lexical index (lexindex), the lex data,
 *      and the miscellaneous parameter table (misc).
 *      There are at most MAXLEX lexes, and at most NRULE sets of
 *      rule data associated with each one.
 */
#define MAXLEX 100
#define NRULE 2
/*
 * 1.1.1  The lexical character array
 */
char lex[MAXLEX][2];
int nlexes;
/*
 *       This contains the lexes as fixed-length 2-character
 *       strings, padded with ' ' if necessary.
 *       The array terminates when lexind = nlexes.
 *       It is searched using int find(ch1, ch2).
 *       The lexes are numbered 0, 1, 2, ...  according to their
 *       position in the array.
 *
 * 1.1.2  The lexical index
 */
int *lexindex[MAXLEX][NRULE];
/*
 *       Given a lex number lexind and a rule table index ruleind,
 *       lexindex[lexind][ruleind] is the address of the
 *       data for that lex in the appropriate set of rules.
 *
 * 1.1.3  The lex data
 *
 *       This is a contiguous block of integers.
 *       Element 0 contains the lex type - there are NLEXTYPS
 *       of these, numbered 0, 1, ..., NLEXTYPS-1.
 */
#define NLEXTYPS 9
/*
 *       The types are
 */
#define EOUTYP  0
#define EOTGTYP 1
#define TGTYP   2
#define FOOTTYP 3
#define WORDTYP 4
#define SYLLTYP 5
#define PHTYP   6
#define DURTYP  7
#define PITTYP  8
/*
 *       In addition, we define a null type (for use
 *       with findlex):
 */
#define NULLTYP -1
/*
 *       For each type, there are nrulelts[lextyp] words of
 *       lex data, so
 *           element 0 is lextyp
 *                   1
 *                   2
 *                  ...
 *               nrulelts[lextyp]-1   is the last word.
 */
int nrulelts[NLEXTYPS];
/*
 *       The lex data which is stored in the rule table is
 *
 *         end-of-utterance     EOUTYP
 *
 *         end-of-tone-group    EOTGTYP
 *
 *         tone group boundary  TGTYP n1 n2 ... n10
 *           [the 10 numbers that define the contour]
 *
 *         foot boundary        FOOTTYP s foot-duration
 *           [s = 0 for unstressed feet, 1 for stressed feet]
 *
 *         word boundary        WORDTYP
 *
 *         syllable boundary    SYLLTYP minlength
 *
 *         phoneme              PHTYP flags mindur av as f1 f2 f3 fh
 *
 *         duration specifier   DURTYP
 *
 *         pitch specifier      PITTYP
 *
 *
 *       The following PHTYP flag bits are defined.
 */
#define VOWEL  000001
#define LVOWEL 000002
#define NASAL  000004
```

```
#define BREATH 000010
#define STOP   000020
#define GLIDE  000040
#define H      000100
#define F3X    000200
#define SONOR VOWEL|NASAL|GLIDE
/*
 *        The phoneme data is addressed symbolically.
 */
#define PHFLAGS 1
#define PHDUR   2
#define PHAV    3
#define PHAS    4
#define PHF1    5
#define PHF2    6
#define PHF3    7
#define PHFH    8
/*
 *
 *
 * 1.1.4  The miscellaneous parameter table
 */
int *misc, *ttmisc,
    *aspmisc, *stopmisc, *iclmisc, *iclfric, *syllmisc, *phmisc, *pitmisc;
/*
 *        Misc[0] is the first miscellaneous parameter,
 *        misc[1] the second, and so on.
 *        grule() sets up syllmisc, ..., to point to appropriate
 *        sub-tables within the miscellaneous parameter table.
 *
 *
 * 1.2  The synthesis list
 *
 *      This is a forward linked list.
 *      Each element has a forward pointer and NCONTENT integers.
 */
#define NCONTENT 5
struct listelt {
    struct listelt *fwdptr;
    int content[NCONTENT];
};
/*
 *      There are three types of list elements;
 *      those which refer to lexes, those which do not, and real-time events.
 *
 * 1.2.1  Lex elements
 *
 *        0 <= content[0] < MAXLEX.
 *        Content[0] contains the index of the lex.
 *        Thus lexindex[content[0]][ruleind] gives the address
 *        of the first word of rule data for the lex
 *        (i.e. points to lextyp).
 *
 * 1.2.2  Non-lex elements
 *
 *        MAXLEX < content[0].
 *        Content[0] identifies the type of the element.
 *        Types are
 */
#define SOUTYP  MAXLEX
#define CLUSTYP MAXLEX+1
/*
 * 1.2.3  Events
 *
 *        As with non-lex elements, content[0] identifies the type
 *        of event.  Event types are negative.
 *        Content[1] gives the event time.
 */
#define TARGTYP -1
#define INCRTYP -2
#define ASPTYP  -3
/*
 *      In all cases, content[1], content[2], ... may contain
 *      information pertinent to the particular element.
 *
 *      lex EOUTYP      content[1] - time of eou
 *
 *      lex EOTGTYP     content[1] - time at end of tg
 *
 *      lex FOOTTYP     content[1] - target foot duration
 *                      content[2] - number of syllables in foot
 *
 *      lex SYLLTYP     content[1] - syllable type:
 *                                       0 - strong stressed
 *                                       1 - weak stressed
 *                                       2 - enclitic
 *                                       3 - proclitic
 *                      content[2] - syllable weight
 *                      content[3] - syllable duration
 *
 *      lex PHTYP       content[1] - time at start of this phoneme
 *                      content[2] - total duration
 *                      content[3] - transition-in duration
 *                      content[4] - 0 if phoneme is 'int'
 *                                   1 if phoneme is 'ext'
 *
 *      lex DURTYP      content[1] - total duration of previous
 *                                   phoneme
 *
 *      lex PITTYP      content[1] - time of pitch specification,
 *                                   relative to start of
 *                                   transition of previous
 *                                   phoneme
 *                      content[2] - value of pitch specification
 *
 *      non-lex CLUSTYP content[1] - type of cluster;
 *                                      0 - C1 cluster
 *                                      1 - N cluster
 *                                      2  - C2 cluster
 *                      content[2] - count of segments in cluster
 */
#define C1CLUS 0
```

```
#define NUCLUS 1
#define C2CLUS 2

/*
 *      event TARGTYP   content[1] - event time
 *                      content[2] - logical parameter number
 *                                   0  av
 *                                   1  as
 *                                   2  f1
 *                                   3  f2
 *                                   4  f3
 *                                   5  fh
 *                                   6  f0
 *                      content[3] - time-increment
 *                      content[4] - parameter target value
 *
 *      event INCRTYP   content[1] - event time
 *                      content[2] - logical parameter number
 *                      content[3] - time-increment during which
 *                                   event is in force
 *                      content[4] - constant parameter-increment
 *                                   throughout event
 *
 *      event ASPTYP    content[1] - event time
 *                      content[2] -
 *                      content[3] - time-increment during which
 *                                   the speech is aspirated
 *
 *
 * 1.5  Segmental synthesis database
 *
 *      Each synthesizer requires two data-bases,
 *      one for the non-real-time segmental synthesis process,
 *      and the other for the real-time device handler.
 */
struct segdata {
    struct listelt *lastp;   /* points into synthesis list, just before
                                last phoneme synthesized */
    struct listelt *last;    /* points into synthesis list, at last
                                phoneme synthesized */
    struct listelt *this;    /* points into synthesis list, at
                                current phoneme */
    struct listelt *next;    /* points into synthesis list, at next
                                phoneme to be synthesized */
    int rules;               /* pointer to the appropriate rule
                                table */
    int sclock;              /* contains time up to which synthesis
                                has been done */
};
/*
 *
 *      NPHYS is number of physical synthesizer parameters to a sample.
 *      NLOG  is number of logical parameters to a sample.
 */
#define NPHYS   8
#define NLOG    7

#define NERRORS 5

struct parmdata {
    int state;           /* set to 0 while synthesizer is inactive,
                                1 while waiting for interrupt,
                                2 while computing next lot of parameters,
                                3 if executing seg from interrupt level,
                                      and waiting for interrupt,
                                4 if executing seg from interrupt level,
                                      and computing next lot of parameters.
                            isp aborts if interrupt occurs when state<=0 or >=4;
                                increments error[0] if interrupt occurs when
                                    state=2 or 4;
                                accepts interrupts which occur when state=1 or 3. */
    int parms[NPHYS]; /* buffer of current parameters */
    struct listelt *parmp;     /* points into synthesis list, at current
                                  dequeueing position */
    int clock;           /* clock value for event dequeueing */
    int error[NERRORS];    /* error[0] - incremented whenever interrupt
                                         occurs with state = 2 or 4,
                              error[1] - incremented whenever an overdue
                                         event is encountered in the
                                         synthesis list,
                              error[2] - incremented whenever a stray element
                                         is found in the synthesis list,
                              error[3] - incremented whenever an event
                                         with negative or zero increment-time
                                         is encountered,
                              error[4] - incremented whenever patscale
                                         cuts off a parameter value */
    int values[NLOG];       /* contains current logical parm values */
    int rates[2][NLOG];     /* contains current rate-of-change of each
                               logical parm, and the time-increment for
                               which it is in force */
    int incrs[2][NLOG];     /* contains current temporary increment for
                               each logical parm, and the time-increment
                               for which it is in force */
    int asp;             /* contains time-increment for which
                               aspiration is in force */
};
/*
 *
 *      NSYNS is the number of synthesizers
 */
#define NSYNS 3
struct segdata sd[NSYNS];    /* allocate segdata for each synthesizer */
struct parmdata pd[NSYNS];   /* allocate parmdata for each synthesizer */
int syntyp;     /* current synthesizer type */
#define PATTYP 0
#define ICLTYP 1
#define COMTYP 2
/*
 *
 *      Define duration of "^" at beginning and end of each utterance
 */
#define SOUDUR  3
```

```
#define EOUDUR  2
/*
 *      ADVANCLOCK is minimum time by which pseudoclock may lead realclock
 */
#define ADVANCLOCK 25
/*
 *
 *      Interpretation of events.
 *
 *
 *      Parameter target event:
 *
 *          Compute a rate to get from the current parameter value to the
 *          target value in the time-increment.
 *          Starting from the next clock tick, apply this rate at each tick.
 *          Thus if no other events intervene, the target will be
 *          attained at the appropriate time (event-time + time-increment).
 *          When the target is reached, zero the rate so that the
 *          parameter remains constant until another specification.
 *          If another parameter target event for the same parameter
 *          is encountered before this one expires,
 *          all memory of this one is erased.
 *
 *
 *      Parameter increment event:
 *
 *          Starting at the event time, the increment is added
 *          to the current parameter value just prior to output,
 *          up to and including clock tick number
 *          event-time + time-increment - 1.
 *
 *
 *      Asp event:
 *
 *          Starting at the event time, the sound is aspirated,
 *          up to and including clock tick number
 *          event-time + time-increment - 1.
 *
 *
 * 1.3  Space
 *
 *     This is a contiguous area of free space, used for
 *     the lex data, the miscellaneous parameter table, and
 *     the synthesis list.
 *     spacep points to the next free location of space.
 */
#define SPACESIZ 4000
int space[SPACESIZ], *spacep;
/*
/*      The freelist is a linked list of listelts, pointed to by
 *      freep, which have been allocated from space and subsequently
 *      freed.
 */
struct listelt *freep;
/*
 *      It is maintained by struct listelt *alloc(), free(p);
 *
 *
 * 2  MISCELLANEOUS DEFINITIONS
 *
 *    pp is a global pointer to the synlst, used in the prosodic routines.
 *    lpp is the last value of pp (follows along the list, just behind pp).
 *    pr is the rule index used by the prosodic routines.
 */
struct listelt *pp, *lpp;
int pr;
/*
 *      reporterrors says whether to report errors (1) or not (0).
 *      altered by the "@e" directive.
 */
int reporterrors;
/*
 *    omode specifies the output expected from isp.
 *    It can be any of the following.
 */
int omode;
#define OSPEECH 0
#define OSYNLST 1
#define OPARMS  2
/*
 * PAT and ICL device addresses.
 * (COMPUTALKER shares the ICL DRV-11 for the moment).
 */
#define PATADDR 0167750
#define ICLADDR 0167760
#define COMADDR 01677b0
/*
 * structure for accessing devices
 */
struct {
    int csr;
    int writebuf;
    int readbuf;
};

#define INTRUPTA 0100
#define CSR1 02
/*
 * channels for output to PDP-11/45
 */
#define PARALLEL 0
#define SERIAL   1
int in_chan;    /*      Serial or parallel input (either 1 or 0)        */
char cha;       /*      One character of context for uttread()          */
/*
 * priorities
 */
#define HIGH 1
#define LOW  0
#define PHIGH 0200
#define PLOW  0
/*
```

```
 *
 *
 *
 */
#define ever ;;
```

Rule table

This table of rules is sent from the host computer to ISP on start-up.

```
[Rule table
[Note that at least two comment lines (or at least, null lines) are used
[as separators between lexical categories.
[eou
!
[
[eotg
.
;
[
[tg: followed by pitches defining each contour for ruleind 0 and (optionally) 1
[-------------------------------------------
0       0 110    0 -15    0 -40 -20    0  33  50
1       0 110    0 -15    0 -40 -20    0  33  50
1+      0 110   30 -15    0 -65 -20    0  33  50
1-      0 110  -20 -15    0 -25 -20    0  33  50
2       0 140    0 -15  -55  50  20    0  33  25
        0 180    0 -25  -45  45  45    0  33  50 [can optionally specify
2-      0 100    0 -15    0  40  20    0  33  25
        0 100    0 -25    0  45  45    0  33  25 [contours for rulind=1
3       0 110    0 -15  -35  25  -5    0  33  33
        0 100    0 -25  -45  45   0    0  33  33 [table like this
3-      0 100    0 -15    0  25   0    0  33  33
4       0 140  -30 -15   15  25 -25    0  33  75
5       0 110   20 -15  -15 -25  25    0  33  25
[-------------------------------------------
[
[foot: followed by stressed/unstressed flag, then target foot duration
/  0 40
/* 1 40
[
[word
    [two spaces for a word boundary
[
[syll: followed by minimum syllable duration
"  14
[
[ph
[  flags are
[    bit     1  -  VOWEL  (set for vowels)
[    bit     2  -  LVOWEL (set for long vowels)
[    bit     4  -  NASAL  (set for nasals)
[    bit     8  -  BREATH (set for ^)
[    bit    16  -  STOP   (set for plosives)
[    bit    32  -  GLIDE  (set for glides)
[    bit    64  -  H      (set for h)
[    bit   128  -  F3X    (set for all glides except "r")
[
[  durations are derived from work by lHW/DRH at Calgary in the summer of
[  1977.  (SU30, phdur model.)
[
[      flags dur  av   as   f1   f2   f3   fh
[-------------------------------------------------
^          8   6   0    0  490 1480 2500 2500
h         64   6  60    0  490 1480 2500 2500
uh         1   5  60    0  490 1480 2500 2500
a          1   8  60    0  720 1240 2540 2500
e          1   7  60    0  560 1970 2640 2500
i          1   6  60    0  360 2100 2700 2500
o          1   8  60    0  600  890 2600 2500
u          1   6  60    0  380  950 2440 2500
aa         1   9  60    0  750 1750 2600 2500
           1   9  60    0  500 1500 2500 2500 [can optionally specify
ee         3   9  60    0  290 2270 3090 2500
           3   9  60    0  250 2000 3000 2500 [values for rulind=1
er         3  15  60    0  580 1380 2440 2500
           3  15  60    0  500 1000 2000 2500 [table like this
ar         3  12  60    0  680 1080 2540 2500
aw         3  13  60    0  450  740 2640 2500
uu         3   7  60    0  310  940 2320 2500
r         32   4  60    0  240 1190 1550 2500
w        160   5  60    0  240  650 2500 2500
l        160   6  60    0  380 1190 3000 2500
y        160   6  60    0  240 2270 3000 2500
m          4   6  40    0  190  690 2000 2500
b         16   7  20    0  100  690 2000 2500
p         16   9   0    0  100  690 2000 2500
n          4   6  40    0  190 1780 3300 2500
d         16   5  20    0  100 1780 3300 2500
t         16   8   0    0  100 1780 3300 2500
ng         4   6  40    0  190 2300 2500 2500
g         16   6  20    0  100 2300 2500 2500
k         16   8   0    0  100 2300 2500 2500
s          0   9   0   50  190 1780 3300 6000
z          0   6  40   50  190 1780 3300 6000
sh         0   7   0   60  190 2120 2700 2300
zh         0   5  40   60  190 2120 2700 2300
f          0   8   0   20  190  690 3300 4000
v          0   5  40   20  190  690 3300 4000
th         0   6   0   10  190 1780 3300 5000
dh         0   5  40   10  190 1780 3300 5000
[-------------------------------------------------
[
[dur
dd   [eg "ar dd10 pp0 25 pp5 30" is like "AR D10 P0 25 P5 30"
[
[pit
```

```
pp
[
[miscellaneous parameter table
[
[ttmisc:  parameters for computing phoneme transition times
   4   2        [...<glide>  or  <vowel> <vowel>:  min total dur, and
                [steady-state time
   4   2        [...<stop or nasal>:  min total dur, and transition time
   4   2        [<stop or nasal>...:  min total dur, and transition time
   5   4        [otherwise:  min total dur, and transition time
[aspmisc:  duration of aspiration after an unvoiced stop
   2            [if stop isn't syllable-initial
   3            [if stop is syllable-initial
   4            [if stop is stressed-syllable-initial
   6            [if stop is tonic-stressed-syllable-initial
   0            [if stop is preceded by an "s" in C1 cluster
[stopmisc:  value of as burst on stop release
 60
[iclmisc:  sound type specifications for icl synthesizer in terms of fh
  6000          [s
  4000          [f
  5000          [th
-10000          [h
  2300          [sh
[iclfric:  parameters for adjusting the icl fricative amplitude.
[set to 0 for maximum; increase to decrease friction.
  5             [for unvoiced fricatives
  2             [for voiced fricatives
[
[syllmisc: syllable weights
[-------------------------------------------------
 50  50         [ <strong> <enclitic>
 67  33         [ <strong> <proclitic>
 33  67         [ <weak>   <enclitic>
150  50 100     [ <strong> <enclitic>  <enclitic>
100 150  50     [ <strong> <enclitic>  <proclitic>
150  75  75     [ <strong> <proclitic> <proclitic>
100 100 100     [ <weak>   <enclitic>  <enclitic>
 50 150 100     [ <weak>   <enclitic>  <proclitic>
[-------------------------------------------------
[
[phmisc:  phoneme expansion/contraction bounds
[(see "A flexible scheme for assigning timing and pitch ...")
  0     [minimum offset for int phoneme (unmarked feet)
  0     [maximum offset for int phoneme (unmarked feet)
100     [minimum factor for int phoneme (%) (unmarked feet)
100     [maximum factor for int phoneme (%) (unmarked feet)
  0     [minimum offset for ext phoneme (unmarked feet)
  0     [maximum offset for ext phoneme (unmarked feet)
100     [minimum factor for ext phoneme (%) (unmarked feet)
100     [maximum factor for ext phoneme (%) (unmarked feet)
  0     [minimum offset for int phoneme (marked feet)
  1     [maximum offset for int phoneme (marked feet)
100     [minimum factor for int phoneme (%) (marked feet)
100     [maximum factor for int phoneme (%) (marked feet)
  0     [minimum offset for ext phoneme (marked feet)
  8     [maximum offset for ext phoneme (marked feet)
100     [minimum factor for ext phoneme (%) (marked feet)
100     [maximum factor for ext phoneme (%) (marked feet)
[
[maximum rate of change of pitch allowed when highlighting pretonic feet
 10     [msec for a 1 Hz decline in pitch
```

Program code

The ISP program follows, beginning with the main program.

```
#include "isp.h"

main()
{
    struct listelt *p;
    int rulerror;

    for (ever) {
        init();
        clrparbuf();
        rulerror = 0;
        if(grules()){
            if(rulerror++ == 4)abortit("Failed to find rules 4 times.\r\n");
            strout("Trying to read the rules again.\r\n");
            continue;
            }
        while ((p = uttread()) != 0) {
            rhythm(p);
            pitch(p);
            if (omode == OSYNLST){
                prsynlst(p);
                jputchar('\04',PARALLEL);
            }
            else {
                seginit(p);
                if (omode == OPARMS){
                        while (pparm(syntyp));
                        jputchar('\04',PARALLEL);
                }
                else {
                    pd[syntyp].state = 1;
                    if (syntyp == PATTYP) PATADDR->csr =| INTRUPTA;
                    else if (syntyp == ICLTYP) ICLADDR->csr =| INTRUPTA;
                    else if (syntyp == COMTYP) {
                        COMADDR->writebuf = 15*(1<<8) + 255;  /* let COMPUTALKER rip */
                        COMADDR->csr =| INTRUPTA;
                    }
                    else abortit("main: illegal synthesizer type");
```

```
        }
    }
}

init()
{
    int i, j;
    syntyp = PATTYP;
    omode = OSPEECH;
    reporterrors = 1;
    pr = 0;
    in_chan = 1;
    cha = ' ';
        PATADDR->csr =& ~INTRUPTA;
        ICLADDR->csr =& ~INTRUPTA;
        COMADDR->csr =& ~INTRUPTA;
        COMADDR->writebuf = 15*(1<<8) + 0;  /* shut up COMPUTALKER */
    for (i = 0; i < NSYNS; i++) {
        pd[i].state = 0;
        for (j = 0; j < NPHYS; j++) pd[i].parms[j] = 0;
        for (j = 0; j < NERRORS; j++) pd[i].error[j] = 0;
    }
}
char *strule "rule table";
clrparbuf(){
int letter;
char c;
letter = 0;
while(letter<10){
    if(((c = getdr(0)) | 040) != strule[letter++]){
        letter = 0;
        if(c == '\0')strout("EOF found clearing DR buffer\n");
        }
    }
while(getdr(0) != '\n');
strout("DR cleared\r\n");
}



#define NMISC     NTTMISC+NASPMISC+NSTOPMISC+NICLMISC+NICLFRIC+NSYLLMISC
                  +NPHMISC+NPITMISC
#define NTTMISC     8
#define NASPMISC    5
#define NSTOPMISC   1
#define NICLMISC    5
#define NICLFRIC    2
#define NSYLLMISC  21
#define NPHMISC    16
#define NPITMISC    1

/*
 * Read in the rule table, and re-initialize space and free list.
 * Rule table comprises a file called "rule", with
 *   (a) a list of lexes and their associated data,
 *       sorted into types and ordered by the type number,
 *       with at least one blank or comment line between
 *       each type;
 *   (b) NMISC miscellaneous parameters, arbitrarily disposed
 *       w.r.t. lines.
 */
grules()
{
    char c;
    int lextyp, lexind, rulei, i, firstlex;

    in_chan = PARALLEL;
    spacep = space;
    freep = 0;
    lextyp = lexind = -1;
/* read in lexes and lex data */
    for (ever) switch (c = chin(in_chan)) {
    case '\0':
        strout("premature end of 'rule' file.\r\n");
        return(1);
    case '\n':
        if (++lextyp >= NLEXTYPS) goto miscs;
        while ((c = chin(in_chan)) == '\n')
        firstlex=1;
    default:
        rulei = -1;
        lex[++lexind][0] = c;
        c = chin(in_chan);
        lex[lexind][1] = c == '\n' ? ' ' : c;
    case ' ':
        rulei = ++rulei < NRULE ? rulei : NRULE-1;
        for (i = rulei; i < NRULE; i++) lexindex[lexind][i] = spacep;
        *spacep++ = lextyp;
        if (c != '\n') spacep =+ i = intsin(spacep);
        else i=0;
        if (firstlex == 1) nrulelts[lextyp] = i+1;
        firstlex = 0;
        if (nrulelts[lextyp] != i+1){
            strout("incorrect number of lex data items in 'rule'\r\n");
            return(1);
        }
        if (lextyp > MAXLEX-2){
            strout("too many lexes in 'rule'\r\n");
            return(1);
        }
    }
/* read the miscellaneous parameters */
miscs:
    nlexes = ++lexind;
    lex[lexind][0] = 0; /* terminator used by findlex */
    misc = spacep;
    while(spacep-misc < NMISC) spacep =+ intsin(spacep);
    if (spacep - misc != NMISC){
        strout("incorrect number of misc parameters in 'rule' file\r\n");
        return(1);
```

```
    }
/* set up globals for accessing misc table */
    ttmisc = misc;
    aspmisc = ttmisc + NTTMISC;
    stopmisc = aspmisc + NASPMISC;
    iclmisc = stopmisc + NSTOPMISC;
    iclfric = iclmisc + NICLMISC;
    syllmisc = iclfric + NICLFRIC;
    phmisc = syllmisc + NSYLLMISC;
    pitmisc = phmisc + NPHMISC;
    in_chan = SERIAL;
    strout("rules read ok\r\n");
    return(0);
}



/*
 * Read an utterance and store it as a synthesis list,
 * returning ptr.
 * If zero is returned, restart isp.
 */
struct listelt *uttread()
{
    struct listelt *syntop;
    char ch1;
    int i;

    pp = syntop = allocit();
    pp->content[0] = SOUTYP; /* first element of synlst is SOUTYP */
    for (ever) {
        adchar(&ch1, &cha);
        if (ch1 == '\0') abortit("read: eof received on input");
        else if (ch1 == '@') {
            adchar(&ch1, &cha);
            switch (ch1) {
            case 's':
                return (0); /* restart isp */
            case 'r':
                if ((i = intin(&cha)) < 0 || i >= NRULE)
                    strout ("illegal rule table index\r\n");
                else pr = i;
                break;
            case 'o':
                omode = intin(&cha);
                syntyp = intin(&cha);
                break;
            case 'm':
                if(( cha = chin(in_chan)) == 's') || ( cha == 'p')){
                        if(cha == 's') in_chan = SERIAL;
                        else in_chan = PARALLEL;
                        }
                else {
                        strout("  Error: s or p modes only.\r\n");
                        }
                if(in_chan == SERIAL)strout("   SERIAL");
                else strout("   PARALLEL");
                jputchar('\r',SERIAL);
                jputchar('\n',SERIAL);
                cha = ' ';
                break;
            case 'e':
                reporterrors = intin(&cha);
                break;
            default:
                strout("illegal directive\r\n");
            }
            i = -1;
        }
        else if ((i = findlex(ch1, cha)) != -1)  {
            if (cha != ' ') adchar(&ch1, &cha);
        }
        else if (ch1 != ' ' && ch1 != '\r' && ch1 != '\n' && ch1 != '\04'
                            && (i = findlex(ch1, ' ')) == -1) {
            strout("\r\ncannot identify ");
            jputchar(ch1,SERIAL);
            jputchar(' ',SERIAL);
            intout(ch1,SERIAL);
            jputchar('\r',SERIAL);
            jputchar('\n',SERIAL);
        }
        if (i != -1) {
            pp = pp->fwdptr = allocit();
            pp->content[0] = i;
            switch (lexindex[i][pr][0]) {
            case SOUTYP:
                return (syntop);
            case DURTYP:
                pp->content[1] = intin(&cha);
                break;
            case PITTYP:
                pp->content[1] = intin(&cha);
                pp->content[2] = intin(&cha);
            }
        }
    }
}

/*
 * Finds the two-character string ch1, ch2 in array lex,
 * returning the lexindex of the string
 * (-1 if not found).
 */
int findlex(ch1, ch2)
char ch1, ch2;
{
    int lexind;
    char c;
    for (lexind = 0; ; lexind++) {
        if ((c = lex[lexind][0]) == '\0') return (-1);
        if (c == ch1 && ch2 == lex[lexind][1]) return (lexind);
```

```
    }
}

/*
 * Advances the characters pointed to by cp1, cp2
 * by reading from the standard input device.
 */
adchar(cp1, cp2)
char *cp1, *cp2;
{
    *cp1 = *cp2;
    *cp2 = chin(in_chan);
}

/*
 * Allocate an element of the segment list,
 * with its fwd ptr zeroed.
 */
struct listelt *allocit()
{
    struct listelt *t;
    int i, prior;

    prior = setprior(HIGH);
    if (freep) {  /* get space from free list */
        t = freep;
        freep = freep->fwdptr;
    }
    else {
        t = spacep++;  /* get space from pool */
        if ((spacep += NCONTENT) >= space + SPACESIZ) {
            setprior(prior);
            abortit("cannot allocate list element");
        }
    }
    t->fwdptr = 0;
    for (i = 0; i < NCONTENT; i++)  t->content[i] = 0;
    setprior(prior);
    return (t);
}

/*
 * Put an element on the free list.
 * Returns old fwd ptr of freed element.
 */
struct listelt *freeit(p)
struct listelt *p;
{
    struct listelt *q;
    int prior;

    prior = setprior(HIGH);
    q = p->fwdptr;
    p->fwdptr = freep;
    freep = p;
    setprior(prior);
    return (q);
}



/*
 * Prints the synthesis list in crude form.
 */
prsynlst(p)
struct listelt *p;
{
    int i, j, end;
    for (end=0; end != 1; p = freeit(p)) {
            if ((i = p->content[0]) < MAXLEX && i >= 0) {
                jputchar(lex[i][0],PARALLEL);
                jputchar(lex[i][1],PARALLEL);
                if (lexindex[i][pr][0] == EOUTYP) end = 1;
            }
            else intout(i,PARALLEL);
            for (i = NCONTENT; i > 1 && p->content[i-1] == 0; i--);
            for (j = 1; j < i; j++)  {
                jputchar(' ',PARALLEL);
                intout(p->content[j],PARALLEL);
            }
            jputchar('\n',PARALLEL);
    }
}


/*
 * Prints the parameter tracks for synthesis of synthesizer i.
 * Returns the state of the synthesizer.
 * Call pparm(i) repeatedly while this state is non-zero
 * to print all the parms for the utterance.
 */
pparm(i)
int i;
{
    int j;

    for (j=0; j < NPHYS; j++) {
        intout(pd[i].parms[j],PARALLEL);
        jputchar(' ',PARALLEL);
    }
    jputchar('\n',PARALLEL);
    switch(i) {
    case PATTYP:
            if ((j = patparm()) == 0) jputchar('\n',PARALLEL);
        else if (sd[PATTYP].sclock-pd[PATTYP].clock < ADVANCLOCK) seg(PATTYP);
            return(j);

    case ICLTYP:
            if ((j = iclparm()) == 0)  jputchar('\n',PARALLEL);
        else if (sd[ICLTYP].sclock-pd[ICLTYP].clock < ADVANCLOCK) seg(ICLTYP);
```

```
                return(j);

    case COMTYP:
        if ((j = comparm()) == 0) jputchar('\n',PARALLEL);
        else if (sd[COMTYP].sclock-pd[COMTYP].clock < ADVANCLOCK) seg(COMTYP);
        return(j);

    default:
                abortit("pparm: does not cater for this synthesizer");
    }
}



/*
 * Reads a char from channel
 * but passes over comments ("\[" to end of line)
 */
char chin(channel)
int channel;
{
    char c;
    if ((c = jgetchar(channel)) == '[')   while ((c = jgetchar(channel)) != '\n');
    return(c=='\t' ?  ' ' : c);
}

/*
 * Reads a signed integer from in_chan, returning 0 if one is not there.
 * Call like  n = intin(&ch)  where ch is initialized as
 * one character of forward context (it is updated by intin
 * accordingly).
 */
int intin(cp)
char *cp;
{
    int n, minus;

    n = minus = 0;
    while (*cp == ' ') *cp = chin(in_chan);
    if (*cp == '-') {
        minus++;
        *cp = chin(in_chan);
    }
    while (*cp >= '0' && *cp <= '9') {
        n = 10*n + *cp - '0';
        *cp = chin(in_chan);
    }
    if (minus) n = -n;
    return(n);
}

/*
 * Reads a list of integers from in_chan, until '\n',
 * and places them in *p++,
 * returning number of integers read.
 * Call-by-value means that the actual parameter p is not updated.
 * Aborts if eof encountered on a line by itself.
 * Stops reading on error (illegal character).
 */
intsin(p)
int *p;
{
    int nints;
    char c;
    c = chin(in_chan);
    for (nints = 0; ; nints++) {
        while (c == ' ') c = chin(in_chan);
        if (nints == 0 && c == '\0') abortit("eof encountered in 'intsin'");
        if ((c < '0' || c > '9') && c != '-')  return(nints);
        *p++ = intin(&c);
    }
}

/*
 * Prints character string, with no newline
 */
strout(p)
char *p;
{
    while (*p != '\0') jputchar(*p++,SERIAL);
}

/*
 * Prints message and aborts
 */
abortit(p)
char *p;
{
    strout(p);
    jputchar('\n',SERIAL);
    jputchar('\r',SERIAL);
    if(in_chan == PARALLEL)strout("Note: input is PARALLEL.");
    else strout("Note: input is SERIAL.");
    jputchar('\n',SERIAL);
    jputchar('\r',SERIAL);
    abort();
}

/*
 * Prints signed integer in decimal on channel
 */
intout(n,channel)
int n,channel;
{
    int flag, k, d;

    if (n < 0) {
        jputchar('-',channel);
        n = -n;
    }
```

```
        pp = lpp->fwdptr;
    case SYLLTYP:
        goto syllb;
    }
}
```

```
/*
 * Counts the phoneme segments in a cluster and updates content[2]
 * of the record.
 */
countclus()
{
    struct listelt *pclus, *lpclus;
    int i;

    pclus = pp;
    lpclus = lpp;
    for (i = 0; afind(PHTYP,CLUSTYP,SYLLTYP) == PHTYP; i++);
    pclus->content[2] = i;
    if (i == 0) lpclus->fwdptr = freeit(pclus);
}
```

```
/*
 * Procedure to classify the syllable types in a foot,
 * and fill in the syllable weights.
 * Call with pfoot pointing to foot.
 */
classsyll(pfoot)
struct listelt *pfoot;
{
    struct listelt *psyll;
    int i, count, type, types;

    pp = pfoot;
    while ((i = afind(SYLLTYP,FOOTTYP,NULLTYP)) == FOOTTYP)
        lpp->fwdptr = freeit(pp); /* no syll in foot - remove foot bdry */
    if (i == EOUTYP) return;
    psyll = pp;
    psyll->content[1] = types = weaksyll();
    psyll->content[2] = types==0 ? 200:100;  /* default syllable weight */
    pp = psyll;
    type = 2;
    count = 1;
    while ((i = afind(SYLLTYP,WORDTYP,FOOTTYP)) != EOUTYP && i != FOOTTYP) {
        if (i == WORDTYP) type = 3;
        else {
            pp->content[1] = type;
            pp->content[2] = 100;  /* default syllable weight */
            types =| ((type == 2) << count++);
/* types contains bits set for weak or enclitic syllables, */
/* e.g.  110     <-->  <type=2> <type=2> <strong> */
/*             <-->  <strong> <enclitic> <enclitic> */
        }
    }
    pfoot->content[2] = count;
    i = -1;
    if (count == 2) switch(types) {
    case 2:
        i = 0;  /* <strong> <enclitic> */
        break;
    case 0:
        i = 2;  /* <strong> <proclitic> */
        break;
    case 3:
        i = 4;  /* <weak> <enclitic> */
        break;
    }
    else if (count == 3) switch (types) {
    case 6:
        i = 6;  /* <strong> <enclitic> <enclitic> */
        break;
    case 2:
        i = 9;  /* <strong> <enclitic> <proclitic> */
        break;
    case 0:
        i = 12; /* <strong> <proclitic> <proclitic> */
        break;
    case 7:
        i = 15; /* <weak> <enclitic> <enclitic> */
        break;
    case 3:
        i = 18; /* <weak> <enclitic> <proclitic> */
        break;
    }

/* overwrite syllable weights with numbers from syllmisc table */
    if (i >= 0) for (pp = pfoot; afind(SYLLTYP,FOOTTYP,NULLTYP) == SYLLTYP;
                                        pp->content[2] = syllmisc[i++]);
}
```

```
/*
 * Calculate the syllable durations for the foot.
 */
syllduns(pfoot)
struct listelt *pfoot;
{
    int total, min, incr, dur, syllex;

    total = 0;
    min = 1000;
```

```
    dur = pfoot->content[1];
    for (pp = pfoot; afind(SYLLTYP,FOOTTYP,NULLTYP) == SYLLTYP;
                                        total =+ pp->content[2]) {
        min = pp->content[2] < min ? pp->content[2] : min;
        sylllex = pp->content[0];
    }
    if ((incr = lexindex[syllex][pr][1] - (dur * min)/total) < 0) incr = 0;
    for (pp = pfoot; afind(SYLLTYP,FOOTTYP,NULLTYP) == SYLLTYP;
                    pp->content[3] = incr + (dur * pp->content[2])/total);
}



/*
 * Calculate the segment durations for the syllable.
 * (s is unmarked/marked status of the current foot.)
 */
/*
 * INT and EXT must be defined as follows:
 */
#define INT 0
#define EXT 1
phdurs(psyll,s)
struct listelt *psyll;
int s;
{
    int sylldur, count, type, stop, dur, discrep, i, j, k,
        number[2], offset[2], factor[2];
    struct listelt *p;

    s = (s!=0)<<3;
    pp = psyll;
    sylldur = pp->content[3];
    afind(CLUSTYP,NULLTYP,NULLTYP);
    if (pp->content[1] == C1CLUS)
        while (afind(PHTYP,CLUSTYP,NULLTYP) == PHTYP) pp->content[4] = INT;
    count = pp->content[2];
    while ((type=afind(PHTYP,CLUSTYP,SYLLTYP)) == PHTYP) {
        pp->content[4] = INT;
        if (count>1 || (lexindex[pp->content[0]][pr][PHFLAGS]&LVOWEL)) {
            pp->content[4] = EXT;
            count = 0;
        }
    }
    stop = 0;
    if (type==CLUSTYP) {
        while (afind(PHTYP,SYLLTYP,NULLTYP)==PHTYP) {
            pp->content[4] = INT;
            p = pp;
            if (stop==0 && (lexindex[pp->content[0]][pr][PHFLAGS]&STOP)) {
                pp->content[4] = EXT;
                stop = 1;
            }
        }
        if (stop==0) p->content[4] = EXT;
    }
    pp = psyll;
    number[INT] = 0;  number[EXT] = 0;  dur = 0;
    while (afind(PHTYP,SYLLTYP,NULLTYP)==PHTYP) {
        number[pp->content[4]]++;
        dur =+ (pp->content[2] = lexindex[pp->content[0]][pr][PHDUR]);
    }
    for (i=EXT; i>=0; i--) {
        discrep = sylldur - dur;
        j = (discrep + (number[i]>>1))/number[i];
        k = s + (i<<2);
        if (j < phmisc[k]) j = phmisc[k];
        else if (j > phmisc[++k]) j = phmisc[k];
        offset[i] = j;
        dur =+ j*number[i];
    }
    discrep = (sylldur*100 + (dur>>1))/dur;
    for (i=EXT; i>=0; i--) {
        k = s + (i<<2) + 2;
        j = discrep;
        if (j < phmisc[k]) j = phmisc[k];
        else if (j > phmisc[++k]) j = phmisc[k];
        factor[i] = j;
    }
    pp = psyll;
    while (afind(PHTYP,SYLLTYP,NULLTYP)==PHTYP) {
        type = pp->content[4];
        pp->content[2] = ((pp->content[2] + offset[type])*factor[type] + 50)/100;
    }
}



/*
 * Fixes up explicitly specified phoneme durations
 * and computes transition times.
 */
phdurcheck(syntop)
struct listelt *syntop;
{
    int dur, oldflag, newflag, tt, i, time;
    struct listelt *p;

    pp = syntop;
    time = 0;
    oldflag = -1;
    while (afind(PHTYP,NULLTYP,NULLTYP) == PHTYP) {
/* fix up explicitly specified phoneme durations */
        if ((i = nextdur()) >= 0) pp->content[2] = i;

/* compute transition time */
        dur = pp->content[2];
        newflag = lexindex[pp->content[0]][pr][PHFLAGS];
```

```
    flag = 0;
    for (k = 10000; k; k = k/10) {
        d = n/k;
        n =% k;
        if (d || flag) {
            flag++;
            jputchar('0' + d,channel);
        }
    }
    if (flag == 0) jputchar('0',channel);
}
```

```
/*
 * High-level rhythm assignment routine.
 * Computes phoneme durations and transition times.
 */
rhythm(syntop)
struct listelt *syntop;
{
    struct listelt *pfoot, *psyll;
    int i, s, sfoot, smark;

/* separate syllables and classify the clusters */
    syllsplit(syntop);
    for (pp = syntop; afind(CLUSTYP,NULLTYP,NULLTYP) == CLUSTYP; pp = psyll->fwdptr) {
        psyll = lpp;  /* pp elt may have been deleted */
        countclus();
    }

/* insert initial foot boundary if necessary */
    pp = syntop;
    s = 0;
    if (afind(FOOTTYP,SYLLTYP,NULLTYP) == SYLLTYP) {
        insert(0);
        s = 1;
    }
    for (i = 0; i < nlexes; i++)
/* find foot lexind and make sfoot equal to stressed foot lexind */
        if (lexindex[i][pr][0] == FOOTTYP)  {
            if (s == 1) pp->content[0] = i;
            if (lexindex[i][pr][1] == 0) s = 0;
            else sfoot = i;
        }
    smark = 0;  /* smark says whether a stressed foot has been seen in this tg */

/* for each foot ... */
    for (pp = syntop; afind(FOOTTYP,NULLTYP,NULLTYP) == FOOTTYP;
                                                    pp = pfoot) {
        pfoot = pp;
        if (smark==0) smark = lexindex[pfoot->content[0]][pr][1];
        if (afind(FOOTTYP,TGTYP,EOTGTYP) != FOOTTYP) {
/* make last foot of tg stressed if appropriate */
            if (smark==0) pfoot->content[0] = sfoot;
            smark = 0;
        pp = pfoot;
/* make last foot of utt stressed (even if another one is stressed as well) */
        if (afind(FOOTTYP,EOUTYP,NULLTYP) == EOUTYP)
            pfoot->content[0] = sfoot;
        pp = pfoot;
/* . assign foot duration, ... */
        if ((pfoot->content[1] = nextdur()) < 0)
            pfoot->content[1] = lexindex[pfoot->content[0]][pr][2];
/* . classify the syllables, ... */
        classsyll(pfoot);
/* . and compute their durations */
        syllduns(pfoot);

/* for each syllable ... */
        s = lexindex[pfoot->content[0]][pr][1];
        for (pp = pfoot; afind(SYLLTYP,FOOTTYP,NULLTYP) == SYLLTYP;
                                                        pp = psyll) {
            psyll = pp;
/* . fix up explicitly specified syllable durations ... */
            if ((i = nextdur()) >= 0) psyll->content[3] = i;
/* . and compute the phoneme durations */
            phdurs(psyll,s);
        }
    }

/* put in initial and final silences */
    for (i = 0; lexindex[i][pr][0] != PHTYP || (lexindex[i][pr][PHFLAGS]&BREATH)==0;
                                                        i++);
    pp = syntop;
    adpp();
    insert(i);
    pp->content[2] = SOUDUR;
    while (pp->content[0] != EOUTYP) adpp();
    insert(i);
    pp->content[2] = EOUDUR;

/* check the phoneme durations and compute transition times */
    phdurcheck(syntop);
}
```

```
/*
 * Syllabification routine.
 * Looks for a minimum of sonority between two peaks, and if there
 * is not a syllable boundary there already,
 * puts one in at an appropriate place.
 * The "appropriate place" is determined by the number of consonants
 * which have the same, minimum, sonority:
 * if there is only one, the boundary is placed before it;
 * if there are two, the boundary is placed between them;
 * if there are three or more, the boundary is placed after the
```

```
 * first two.
 * Always places a boundary directly before a phoneme
 * (so as not to misplace any DURTYP elements).
 * Also marks the start of the C1, NU, C2 parts of a syllable with
 * a CLUSTYP element, if the appropriate part exists.
 * Routine relies on an EOUTYP element being returned continually
 * by adpp, at the end of the utterance.
 */
syllsplit(syntop)
struct listelt *syntop;
{
    int i, j, c, son, sylllex, nph;
    struct listelt *p, *q, *lp;

/* set sylllex to index of syllable boundary lex */
    for (sylllex = 0; lexindex[sylllex][pr][0] != SYLLTYP; sylllex++);

/* pre-scan synlst to put syllable boundaries at word boundaries and */
/* either side of ^'s */
    pp = syntop;
prescan:
    switch (afind(PHTYP,WORDTYP,NULLTYP)) {
    case PHTYP:
        if ((lexindex[pp->content[0]][pr][PHFLAGS] & BREATH) == 0) goto prescan;
        insert(sylllex);
        adpp();
    case WORDTYP:
        if (afind(PHTYP,SYLLTYP,NULLTYP) == PHTYP) insert(sylllex);
        goto prescan;
    }

    pp = syntop;
    nph = 1;
/* here on syllable boundary */
syllb:
    switch(find(PHTYP,SYLLTYP,NULLTYP)) {
    default:
    exit:
/* remove redundant terminal syllable boundary */
        if (nph == 0) lp->fwdptr = freeit(lp->fwdptr);
        return;
    case PHTYP:
        insert(sylllex);  /* create syll boundary */
    case SYLLTYP:
        son = 0;
        lp = lpp;
        nph = 0;
    }

/* search for maximum sonority, filling in cluster elements */
    i = 0;  /* set when it is determined that a C1 or NU cluster exists */
smax:
    switch(c = afind(PHTYP,SYLLTYP,NULLTYP)) {
    case PHTYP:
        nph++;
        if ((j = updson(&son)) < 0) break;
        if (i == 0) {
            insert(CLUSTYP);  /* create C1CLUS elt */
            pp->content[1] = C1CLUS;
            i++;
            p = lpp;
            adpp();
        }
/* keep p pointing just before first segment with max son */
        else if (j > 0) p = lpp;
        goto smax;
    case SYLLTYP:
        lp = lpp;
        nph = 0;
        if (son == 0)  {
            lpp->fwdptr = freeit(pp);
            pp = lpp;
            goto smax;
        }
        pp = lpp;
    }
    if (c == EOTGTYP && nph == 0) goto exit;
/* create NU-type cluster element */
    if ((p->fwdptr)->content[0] != CLUSTYP)  {
        q = p->fwdptr;
        p = p->fwdptr = allocit();
        p->fwdptr = q;
    }
    else p = p->fwdptr;
    p->content[0] = CLUSTYP;
    p->content[1] = NUCLUS;
    if (c == EOUTYP) goto exit;

/* search for minimum sonority and mark appropriate place */
    i = 0;  /* set when it is determined that a C2 cluster exists */
newmin:
    c = 1;
    if (son == 0 && i == 0) {
        insert(CLUSTYP);  /* create C2 cluster element */
        pp->content[1] = C2CLUS;
        i++;
    }
mark:
    p = lpp;
smin:
    switch(afind(PHTYP,SYLLTYP,NULLTYP)) {
    default:
        goto exit;
    case PHTYP:
        nph++;
        if ((j = updson(&son)) < 0) goto newmin;
        if (j == 0) {
            if (++c <= 3) goto mark;
            goto smin;
        }
/* a definite minimum has been reached - make a syll boundary */
        lpp = p;
```

```
        if (oldflag == -1) tt = SOUDUR - 1;
        else {
            if (newflag & (STOP|NASAL)) i = 2;
            else if (oldflag & (STOP|NASAL)) i = 4;
            else if ((newflag & GLIDE) || ((oldflag & VOWEL) && (newflag & VOWEL))) {
                if (dur < ttmisc[0]) dur = ttmisc[0];
                tt = dur - ttmisc[1];
                goto done;
            }
            else i = 6;
            if (dur < ttmisc[i]) dur = ttmisc[i];
            tt = ttmisc[++i];
        }
    done:
        pp->content[1] = time;
        time += pp->content[2] = dur;
        pp->content[3] = tt;
        oldflag = newflag;
    }
    pp->content[1] = time;
}
```

```
/*
 * Decides whether a syll is weak (returns 1) or strong (returns 0).
 * (^ sylls are weak.)
 * Call with pp pointing to syll.
 */
int weaksyll()
{
    int i;
    while (afind(CLUSTYP,NULLTYP,NULLTYP) == CLUSTYP && pp->content[1] != NUCLUS);
    if (pp->content[2] > 1) return(0);
    if (afind(PHTYP,CLUSTYP,NULLTYP)==PHTYP)
        if (lexindex[pp->content[0]][pr][PHFLAGS]&LVOWEL) return(0);
    if (find(CLUSTYP,SYLLTYP,NULLTYP)==CLUSTYP)
        if (pp->content[2] >1) return(0);
    return(1);
}
```

```
/*
 * Updates the sonorance, returning the difference between the new and old values.
 * Call like diff = updson(&son).
 * Prosodic pointer pp is assumed to point to a phoneme.
 */
int updson(p)
int *p;
{
    int son, flags;
    son = *p;
    flags = lexindex[pp->content[0]][pr][PHFLAGS];
    *p = (flags & VOWEL) ? 2 : (flags & ( /* SONOR | */ BREATH)) ? 1 : 0;
    return(*p - son);
}
```

```
#define TONIC 0
#define PRETONIC 1

/*
 * Assigns intonation to each tone group
 */
pitch(syntop)
struct listelt *syntop;
{
    struct listelt *pfoot;
    int tglex, eotglex, feet, nfeet, i, *tgp;

/* find null tg lexind */
    for (tglex = 0; lexindex[tglex][pr][0] != TGTYP; tglex++);

/* find null eotg lexind */
    for (eotglex = 0; lexindex[eotglex][pr][0] != EOTGTYP; eotglex++);

tgstart:
    tgp = &lexindex[tglex][pr][1];
    pp = syntop;
    if ((i = afind(PHTYP,TGTYP,EOTGTYP)) == EOUTYP) return;
    if (i == EOTGTYP) {
        syntop = pp;
        goto tgstart;
    }
    if (i == TGTYP) {
        syntop = pp;
        tgp = &lexindex[pp->content[0]][pr][1];
    }
    pp = syntop;
    if ((i = afind(PITTYP,TGTYP,EOTGTYP)) == PITTYP) {
        afind(TGTYP,EOTGTYP,NULLTYP);
        syntop = lpp;
        goto tgstart;
    }
    else if (i == TGTYP) insert(eotglex);
    pp = syntop;

/* count pre-tonic feet */
    for (feet = 0; afind(FOOTTYP,TGTYP,EOTGTYP) == FOOTTYP &&
        lexindex[pp->content[0]][pr][1] == 0;  feet++);
    i = 0;
    pp = syntop;
    if (feet) {
        if (tgp[0] && afind(FOOTTYP,PHTYP,NULLTYP) == FOOTTYP) {
```

```
            afind(PHTYP,NULLTYP,NULLTYP);
            insertpit(0, tgp[0]);
            i = 1;
        }
        for ( ; i < feet; i++) {
            afind(FOOTTYP,NULLTYP,NULLTYP);
            pfoot = pp;
            intone(pfoot, tgp[1] + (i*tgp[2] + (feet>>1))/feet, tgp[3], tgp[8],
                                                                    PRETONIC);
            pp = pfoot;
        }
/* find last phoneme in pre-tonic */
        while (afind(PHTYP,FOOTTYP,NULLTYP) == PHTYP) syntop = pp;
        pp = syntop;
        insertpit(pp->content[2] - 1, tgp[1] + tgp[2]);
    }
    if (afind(FOOTTYP,TGTYP,EOTGTYP) != FOOTTYP) {
        syntop = lpp;
        goto tgstart;
    }

/* assign pitch to tonic foot */
    i = tgp[1] + tgp[2] + tgp[4];
    syntop = pp;
    intone(pp, i, tgp[6], tgp[9], TONIC);
    i += tgp[5];
    while (afind(FOOTTYP,TGTYP,EOTGTYP) == FOOTTYP) syntop = pp;
    pp = syntop;
    if (tgp[7]) {
        afind(PHTYP,NULLTYP,NULLTYP);
        insertpit(0, i);
        i = tgp[7];
    }
    while (afind(PHTYP,TGTYP,EOTGTYP) == PHTYP) syntop = pp;
    pp = syntop;
    insertpit(pp->content[2] - 1, i);
    goto tgstart;
}
```

```
/*
 * Assigns intonation to a pre-tonic foot or to the tonic foot.
 * Puts pit at the start of the foot, and pit+pincr at
 * percent of the way along it.
 * type is TONIC or PRETONIC to determine whether pitmisc[0] should
 * be used to limit the rate of change of pitch.
 */
intone(pfoot, pit, pincr, percent, type)
struct listelt *pfoot;
int pit, pincr, percent, type;
{
    int s, t, maxpincr;

    pp = pfoot;
    for (t = 0; afind(PHTYP,FOOTTYP,EOTGTYP) == PHTYP; t += pp->content[2]);
    pp = pfoot;
    afind(PHTYP,NULLTYP,NULLTYP);
    insertpit(0, pit);
    if ((s = (percent*t + 50)/100) <= 0 || s >= t) return;
    if (type == PRETONIC)
        if (pincr > (maxpincr=(10*s)/pitmisc[0])) pincr = maxpincr;
        else if (pincr < -maxpincr) pincr = -maxpincr;
    t = 0;
    pp = lpp;
    while(t + pp->content[2] <= s) {
        t += pp->content[2];
        afind(PHTYP,NULLTYP,NULLTYP);
    }
    insertpit(s - t, pit + pincr);
}
```

```
/*
 * Inserts a pitlex after the current phoneme.
 * Call with pp pointing to phoneme.
 */
insertpit(reltime, value)
int reltime, value;
{
    int pitlex;

/* find pitch marker lexind */
    for (pitlex = 0; lexindex[pitlex][pr][0] != PITTYP; pitlex++);

    adpp();
    while (pp->content[0] == pitlex) adpp();
    insert(pitlex);
    pp->content[1] = reltime;
    pp->content[2] = value;
}
```

```
/*
 * Advances pp, keeping lpp up to date.
 * Refuses to advance past end of list.
 * Returns pp.
 */
struct listelt *adpp()
{
    lpp = pp;
    if (pp) pp = pp->fwdptr;
}
```

```
 * Calls adpp() repeatedly (possibly not at all) until an element
 * of type1, type2, type3, or EOUTYP, is encountered.
 * Returns type of element.
 */
int find(type1, type2, type3)
int type1, type2, type3;
{
    int i;
    for ( ;; adpp()) {
        if (pp == 0) return(EOUTYP);
        if ((i = pp->content[0]) < MAXLEX) i = lexindex[i][pr][0];
        if (i == type1 || i == type2 || i == type3 || i == EOUTYP) return(i);
    }
}


/*
 * Like find() but guarantees to advance pp at least once.
 */
int afind(type1, type2, type3)
int type1, type2, type3;
{
    if (pp) adpp();
    return(find(type1, type2, type3));
}


/*
 * Checks to see if the next element in the synthesis list is a DURTYP
 * one.
 * If so, returns the duration and removes the element from the list.
 * If not, returns -1.
 * (Skips over PITTYP elements to avoid them obscuring a subsequent DURTYP.)
 */
int nextdur()
{
    struct listelt *p, *lp;
    int i, j;

    p = pp;
    j = -1;
next:
    lp = p;
    p = p->fwdptr;
    if ((i = p->content[0]) < MAXLEX) {
        if ((i = lexindex[i][pr][0]) == PITTYP) goto next;
        else if (i == DURTYP) {
            j = p->content[1];
            lp->fwdptr = freeit(p);
        }
    }
    return(j);
}

/*
 * inserts an elt at the current point in the synthesis list,
 * making it type 'type'.
 * returns with pp pointing to newly-inserted elt.
 */
insert(type)
int type;
{
    pp = allocit();
    pp->fwdptr = lpp->fwdptr;
    lpp->fwdptr = pp;
    pp->content[0] = type;
}


/*
 * Initializes current synthesizer to say the utterance in
 * the synlist pointed to by p.
 */
seginit(p)
struct listelt *p;
{
    int i, j, t, oldt;
    struct listelt *lp, *s;

/* wait for all synthesizers to become free */
wait:
    for (i = 0; i < NSYNS; i++) if (pd[i].state) goto wait;

/* report any errors in last synthesis */
    for (i = 0; i < NERRORS; i++)  if (pd[syntyp].error[i]) {
        if (reporterrors) {
            jputchar('\n',SERIAL);
            intout(syntyp,SERIAL);
        }
        for (j = 0; j < NERRORS; j++) {
            if (reporterrors) {
                jputchar(' ',SERIAL);
                intout(pd[syntyp].error[j],SERIAL);
            }
            pd[syntyp].error[j] = 0;
        }
        if (reporterrors) jputchar(' ',SERIAL);
    }

/* initialize synthesis data base */
    pd[syntyp].parap = sd[syntyp].lastp = p;
    sd[syntyp].sclock = 0;
    sd[syntyp].rules = pr;
    sd[syntyp].this = 0;
    for (s = p->fwdptr;
    s && ((j = s->content[0]) >= MAXLEX || lexindex[j][pr][0] != PHTYP);
        s = s->fwdptr);
    sd[syntyp].next = s;
    for (i = 0; i < NLOG; i++)
```

```
        pd[syntyp].rates[0][i] = pd[syntyp].incrs[0][i] = 0;
    pd[syntyp].asp = 0;
    pd[syntyp].clock = -2;

/* change PITTYP specifications into pitch TARGTYP events */
/* SOU lex is turned into the initial pitch event */
    lp = 0;
    pp = p;
    oldt = 0;
    while ((i = afind(PHTYP,PITTYP,NULLTYP)) != EOUTYP) {
        if (i == PHTYP) t = pp->content[1];
        else {
            p->content[0] = TARGTYP;
            p->content[1] = oldt;
            p->content[2] = 6;  /* pitch event */
            p->content[3] = t + pp->content[1] - oldt;
            p->content[4] = parmscale(syntyp,6,pp->content[2]);
            oldt = t + pp->content[1];
            lp = lpp;
            p = pp;
        }
    }
    if (lp) lp->fwdptr = freeit(p);

/* start off synthesis */
    while (seg(syntyp) == 0);
}


/*
 * Synthesize some more for synthesizer i.
 * Seg makes sure it isn't called re-entrantly, and returns
 * zero if it hasn't been executed.
 */
int segflag 0;

int seg(i)
int i;
{
    if (segflag) return(0);
    segflag = 1;
    while (sd[i].next && sd[i].sclock - pd[i].clock < 2*ADVANCLOCK) advanceseg(i);
    segflag = 0;
    return(1);
}


/*
 * Main routine to do the segmental synthesis
 * for a phoneme, and make appropriate real-time queue
 * entries in synthesis list.
 */
advanceseg(i)
int i;
{
    struct listelt *s, *p;
    int j, *pl, *pt, *pn, tt;

/* update last, this, next elements of segdata structure */
    sd[i].last = sd[i].this;
    sd[i].this = sd[i].next;
    for (s = sd[i].this->fwdptr;
    s && ((j = s->content[0]) >= MAXLEX || lexindex[j][sd[i].rules][0] != PHTYP);
        s = s->fwdptr);
    sd[i].next = s;

/* frig values in the rule table if             */
/*     (a) this is last phoneme of utterance    */
/*     (b) this is an ^ or an h                 */
/*     (c) this is a glide (but not "r")        */
    if (sd[i].last) pl = &lexindex[sd[i].last->content[0]][sd[i].rules][0];
    else pl = 0;
    pt = &lexindex[sd[i].this->content[0]][sd[i].rules][0];
    if (s) pn = &lexindex[s->content[0]][sd[i].rules][0];
    else pn = 0;
    if (pn == 0) for (j = 0; j < 4; j++) pt[PHF1+j] = pl[PHF1+j];
    else {
        if (pt[PHFLAGS] & BREATH + H) for (j = 0; j < 4; j++) pt[PHF1+j] = pn[PHF1+j];
        else if (pt[PHFLAGS] & F3X) pt[PHF3] = pn[PHF3];
    }

/* now queue the parameter target events for the transition */
    tt = sd[i].this->content[3];
    for (j = 0; j < NLOG-1; j++)
        qsdevent(i, TARGTYP, j, tt, parmscale(i,j,pt[PHAV+j]));

/* if this is voiced stop, put in amplitude event for zero if */
/* phoneme either side is unvoiced */
    if (pt[PHFLAGS] & STOP && pt[PHAV] && (pl[PHAV] == 0 || pn[PHAV] == 0))
        qsdevent(i, TARGTYP, 0, tt, parmscale(i,0,0));

/* if this is h, make it aspirated */
    if (pt[PHFLAGS] & H) qsdevent(i, ASPTYP, -1, sd[i].this->content[2], -1);

/* if last was stop ... */
    if (pl && pl[PHFLAGS] & STOP) {
/* . make fast amplitude transition ... */
        qsdevent(i, TARGTYP, 0, 1, parmscale(i,0,pt[PHAV]));
/* . provided this isn't a stop, make stop burst ... */
        if ((pt[PHFLAGS] & STOP) == 0) {
            qsdevent(i, INCRTYP, 1, 2, parmscale(i,1,stopmisc[0]));
/* . and if last was unvoiced, aspirate it to extent indicated by its */
/*   content[4]+1 (+1 because amplitude will be zero anyway for the */
/*   current sample */
            if (pl[PHAV] == 0) qsdevent(i, ASPTYP, -1,
                                aspmisc[sd[i].last->content[4]] + 1, -1);
        }
    }

/* update sclock */
    if (sd[i].last) sd[i].sclock = sd[i].last->content[1];
```

```
/* now remove all but real-time events from the queue up to this phoneme */
    if ((p = sd[i].last) == 0) return;
    while (p != sd[i].this)
        if (p->content[0] >= 0) sd[i].lastp->fwdptr = p = freeit(p);
        else {
            sd[i].lastp = p;
            p = p->fwdptr;
        }
    if (sd[i].next == 0) {
        sd[i].lastp->fwdptr = freeit(p);
        sd[i].lastp = 0;
    }
}


/*
 * Put an event in the synlist, taking the synlist pointer,
 * the event time, and the rule index, from sd[i].
 */
qsdevent(i, type, j, timeincr, value)
int i, type, j, timeincr, value;
{
    qevent(sd[i].this, sd[i].this->content[1], sd[i].rules, type, j, timeincr, value);
}


/*
 * Put an event in the synlist.
 * Starts searching list from p.
 * Queues events after existing cotemporaneous ones,
 * so that events queued later take precedence.
 */
qevent(p, time, rules, type, i, timeincr, value)
struct listelt *p;
int time, rules, type, i, timeincr, value;
{
    struct listelt *lp;
    int j;

    for (ever) {
        lp = p;
        p = p->fwdptr;
        if ((((j = p->content[0]) < 0 ||
            (j < MAXLEX && ((j = lexindex[j][rules][0]) == PHTYP || j == EOUTYP)))
            && p->content[1] > time)  {
            p = allocit();
            p->fwdptr = lp->fwdptr;
            p->content[0] = type;
            p->content[1] = time;
            p->content[2] = i;
            p->content[3] = timeincr;
            p->content[4] = value;
            lp->fwdptr = p;
            return;
        }
    }
}


/*
 * Scale parameter i, value x, for synthesizer synty
 * (including shift to make scaled value significant enough for
 * rate calculations).
 */
parmscale(synty, i, x)
int synty, i, x;
{
    if (synty == PATTYP) return(patscale(i,x) << 6);
    if (synty == ICLTYP) return(iclscale(i,x) << 5);
    if (synty == COMTYP) return(comscale(i,x) << 6);
    abortit("parmscale: illegal synthesizer type");
}



#define NLOGS 256
int logtable[NLOGS] {
/*  logtable(i) = 2^13 log(i/32 + 1)  */
          0,    364,    716,   1059,   1392,   1716,   2031,   2338,
       2637,   2929,   3214,   3492,   3764,   4029,   4289,   4543,
       4792,   5036,   5274,   5509,   5738,   5963,   6184,   6401,
       6614,   6823,   7029,   7231,   7429,   7625,   7817,   8006,
       8192,   8375,   8556,   8733,   8908,   9081,   9251,   9419,
       9584,   9747,   9908,  10066,  10223,  10378,  10530,  10681,
      10829,  10976,  11121,  11264,  11406,  11546,  11684,  11821,
      11956,  12089,  12221,  12352,  12481,  12609,  12735,  12860,
      12984,  13106,  13228,  13348,  13468,  13586,  13701,  13816,
      13930,  14043,  14155,  14266,  14376,  14485,  14593,  14700,
      14806,  14911,  15015,  15118,  15221,  15322,  15423,  15522,
      15621,  15719,  15817,  15913,  16009,  16104,  16198,  16291,
      16384,  16476,  16567,  16658,  16748,  16837,  16925,  17013,
      17100,  17187,  17273,  17358,  17443,  17527,  17611,  17694,
      17776,  17858,  17939,  18020,  18100,  18179,  18258,  18337,
      18415,  18493,  18570,  18646,  18722,  18798,  18873,  18947,
      19021,  19095,  19168,  19241,  19313,  19385,  19456,  19527,
      19598,  19668,  19738,  19807,  19876,  19944,  20013,  20080,
      20148,  20215,  20281,  20347,  20413,  20479,  20544,  20609,
      20673,  20737,  20801,  20864,  20927,  20990,  21052,  21114,
      21176,  21237,  21298,  21359,  21420,  21480,  21540,  21599,
      21658,  21717,  21776,  21834,  21893,  21950,  22008,  22065,
      22122,  22179,  22235,  22291,  22347,  22403,  22458,  22513,
      22568,  22623,  22677,  22731,  22785,  22838,  22892,  22945,
      22998,  23050,  23103,  23155,  23207,  23259,  23310,  23362,
      23413,  23463,  23514,  23564,  23615,  23665,  23714,  23764,
      23813,  23862,  23911,  23960,  24009,  24057,  24105,  24153,
      24201,  24248,  24296,  24343,  24390,  24437,  24483,  24530,
      24576,  24622,  24668,  24714,  24759,  24805,  24850,  24896,
      24940,  24984,  25029,  25073,  25117,  25161,  25205,  25249,
      25292,  25336,  25379,  25422,  25465,  25508,  25550,  25593,
      25635,  25677,  25719,  25761,  25803,  25844,  25886,  25927};
```

```
/*
 * Logscale returns an n-bit number corresponding to parameter value x
 * logarithmically scaled.
 * Returns  (2^n * log(x/lim0) / log(lim1/lim0).
 * synty is the synthesizer type, for recording out-of-range errors.
 */
int logscale(x, lim0, lim1, n, synty)
int x, lim0, lim1, n, synty;
{
    int low, high, i, d, max;

    max = (1<<n) -1;
    if (lim0 < lim1)    {  low = lim0;    high = lim1;  }
    else                {  low = lim1;    high = lim0;  }
    if (x < low || x >= high) {
        pd[synty].error[4]++;
        if (x < low) x = low;
        else x = high-1;
    }
    if ((i = scalediv(high,5,low) - 32) > NLOGS-1) abortit("log table not big enough");
    d = scalediv(logtable[scalediv(x,5,low)-32], n, logtable[i]);
    if (d > max) {
        pd[synty].error[4]++;
        d = max;
    }
    if (lim0 < lim1) return(d);   else return(max - d);
}

/*
 * scalediv computes (a * 2^n)/b as an integer, to maximum precision.
 */
int ldivr;  /* holds the remainder after ldiv */
int scalediv(a, n, b)
int a, n, b;
{
    register int i;
    i = ldiv(a >> (b-n), a << n, b);
    return(i + (ldivr > (b>>1)));
}
```

```
/*
 * PAT calibration data
 *
 *      A linear scaling is applied to all parameters except f0.
 *      The array patcal[] gives the minimum value and the
 *      maximum value, for each parameter.
 */
int patcal[7][2] {
      0,   64,     /*  av  */
      0,   64,     /*  as  */
    100, 1050,     /*  f1  */
    500, 2500,     /*  f2  */
   1550, 3500,     /*  f3  */
   2000, 7000,     /*  fn  */
     33,  203};    /*  f0  */
int aspfactor 2;  /* divide av by this to get ah1 */
```

```
/*
 * PAT handler.
 * Enter on PAT interrupt.
 */
int patpending 0;    /* incremented whenever an interrupt occurs during output
                        to PAT or during patparm; in which case the interrupt is
                        held pending completion of patparm    */
pat()
{
    register int i, j;

/* check for illegal state */
    if ((i=pd[PATTYP].state) <= 0 || i > 4) abortit("pat: illegal PAT interrupt");
/* ignore the very first interrupt  */
    if(pd[PATTYP].clock <-1) {
        pd[PATTYP].clock++;
        return;
    }
/* check for re-entrancy */
    if (i == 2 || i == 4) if (patpending++) pd[PATTYP].error[0]++;  else;
    else {
        pd[PATTYP].state++;
        patpending++;
        while (patpending) {
            patpending = 0;
            mtps(PLOW);  /* make interruptable */
/* output pd[PATTYP].parms[] to PAT */
            j = 1<<6;
            for (i=0; i<NPHYS; i++) {
                PATADDR->writebuf = pd[PATTYP].parms[i] | j;
                j =<< 1;
            }
/* get next lot of parameters ready */
            i = patparm();
            mtps(PHIGH);
        }
        if (i) {
/* exit to seq if necessary */
            if (pd[PATTYP].state == 2 &&
                 sd[PATTYP].sclock-pd[PATTYP].clock < ADVANCLOCK) {
                pd[PATTYP].state++;
                mtps(PLOW);
                seq(PATTYP);
                mtps(PHIGH);
                pd[PATTYP].state--;
            }
            pd[PATTYP].state--;
        }
        else {
            PATADDR->csr =& ~INTRUPTA;
            pd[PATTYP].state = 0;
```

```
            patpending=0;
        }
    }
}


/*
 * Prepare next lot of parameters for PAT,
 * and see to any current events in the synthesis list.
 * Returns 0 if the end of the synthesis list is encountered; otherwise returns 1.
 */
patparm()
{
    register int i, t, x;
    int e;

    pd[PATTYP].clock++;
/* update current logical parameter values */
    for (i = 0; i < NLOG; i++) {
        pd[PATTYP].values[i] += pd[PATTYP].rates[0][i];
        if (--pd[PATTYP].rates[1][i] == 0) pd[PATTYP].rates[0][i] = 0;
    }

/* dequeue any events, and deal with them */
    for (ever) {
        if (pd[PATTYP].parmp == sd[PATTYP].lastp) {
            pd[PATTYP].error[2]++;
            break;
        }
        if ((x = pd[PATTYP].parmp->content[1] - pd[PATTYP].clock) > 0) break;
        if ((e = pd[PATTYP].parmp->content[0]) == EOUTYP) {
            freeit(pd[PATTYP].parmp);
            return(0);
        }
        if (x < 0) {
            pd[PATTYP].error[1]++;
            pd[PATTYP].clock = pd[PATTYP].parmp->content[1];
        }
        if ((t = pd[PATTYP].parmp->content[3]) <= 0) {
            pd[PATTYP].error[3]++;
abortit("error 3 in pat.c");
            pd[PATTYP].parmp->content[3] = 1; /* set it to a safe value */
        }
        else if (e == ASPTYP) pd[PATTYP].asp = t;
        else {
            i = pd[PATTYP].parmp->content[2];
            x = pd[PATTYP].parmp->content[4];
            if (e == TARGTYP) {
                pd[PATTYP].rates[0][i] = (x - pd[PATTYP].values[i] + (t >> 1))/t;
                pd[PATTYP].rates[1][i] = t;
            }
            else {
                pd[PATTYP].incrs[0][i] = x;
                pd[PATTYP].incrs[1][i] = t;
            }
        }
        pd[PATTYP].parmp = freeit(pd[PATTYP].parmp);
    }

/* compute current parameter values */
    for (i = 0; i < NLOG; i++) {
        pd[PATTYP].parms[i] = patlimit(pd[PATTYP].values[i]
                                + pd[PATTYP].incrs[0][i] + 32 >> 6);
        if (-- pd[PATTYP].incrs[1][i] == 0) pd[PATTYP].incrs[0][i] = 0;
    }

/* shuffle parameter values for PAT */
    x = pd[PATTYP].parms[6];
    pd[PATTYP].parms[6] = pd[PATTYP].parms[5];
    pd[PATTYP].parms[5] = pd[PATTYP].parms[1];
    i = pd[PATTYP].parms[0];
    pd[PATTYP].parms[0] = x;
    if (pd[PATTYP].asp) {
        pd[PATTYP].asp--;
        pd[PATTYP].parms[1] = 0;
        pd[PATTYP].parms[7] = patlimit(i/aspfactor);
    }
    else {
        pd[PATTYP].parms[1] = i;
        pd[PATTYP].parms[7] = 0;
    }

    return(1);
}


/*
 * Scale parameter number i, value x, into a 6-bit quantity
 * according to patcal[][].
 */
int patscale(i, x)
int i, x;
{
    if (i == 6)  return(logscale(x, patcal[6][0], patcal[6][1], 6, PATTYP));

    if (patcal[i][0] == 0 && patcal[i][1] == 64) return(x);

    return(patlimit(scalediv(x-patcal[i][0], 6, patcal[i][1]-patcal[i][0])));
}


/*
 * Limit argument to range [0,63].
 */
int patlimit(x)
int x;
{
    if (x < 0) {
        pd[PATTYP].error[4]++;
        return(0);
    }
    if (x > 63) {
```

```
        pd[PATTYP].error[4]++;
        return(63);
    }
    return(x);
}
```

```
/*
 * ICL physical parameters are
 *      F0
 *      F1
 *      F2
 *      A2
 *      F3
 *      A3
 *      A1
 *      DS
 */
```

```
/*
 * ICL handler.
 * Enter on ICL interrupt.
 */
int iclpending 0;     /* incremented whenever an interrupt occurs during output
                         to ICL or during iclparm; in which case the interrupt
                         is held pending completion of iclparm */
icl()
{
    register int i;

/* temporary frig so that COMPUTALKER can share ICL's DRV-11 */
    if (pd[ICLTYP].state == 0) {
        computalker();
        return;
    }
/* check for illegal state */
    if ((i=pd[ICLTYP].state) <= 0 || i > 4) abortit("icl: illegal ICL interrupt");
/*  ignore the very first interrupt  */
    if(pd[ICLTYP].clock <-1) {
        pd[ICLTYP].clock++;
        return;
    }
/* check for re-entrancy */
    if (i == 2 || i == 4) if (iclpending++) pd[ICLTYP].error[0]++;  else;
    else {
        pd[ICLTYP].state++;
        iclpending++;
        while (iclpending) {
            iclpending = 0;
            mtps(PLOW);  /* make interruptable */
/* output pd[ICLTYP].parms[] to ICL */
            ICLADDR->writebuf = pd[ICLTYP].parms[0];  /* F0 */
            ICLADDR->writebuf = pd[ICLTYP].parms[1];  /* F1 */
            ICLADDR->writebuf = pd[ICLTYP].parms[2]+(pd[ICLTYP].parms[3]<<8);/* A2-F2 */
            ICLADDR->writebuf = pd[ICLTYP].parms[4]+(pd[ICLTYP].parms[5]<<8);/* A3-F3 */
/* amplitude bits are inverted */
            ICLADDR->writebuf = ((~pd[ICLTYP].parms[6])&0777) +
                                        (pd[ICLTYP].parms[7]<<10);  /* DS-A1 */
/* get next lot of parameters ready */
            i = iclparm();
            mtps(PHIGH);
        }
        if (i) {
/* exit to seg if necessary */
            if (pd[ICLTYP].state=2 && sd[ICLTYP].sclock-pd[ICLTYP].clock<ADVANCLOCK){
                pd[ICLTYP].state++;
                mtps(PLOW);
                seg(ICLTYP);
                mtps(PHIGH);
                pd[ICLTYP].state--;
            }
            pd[ICLTYP].state--;
        }
        else {
            ICLADDR->csr =& ~INTRUPTA;
            pd[ICLTYP].state = 0;
            iclpending = 0;
        }
    }
}
```

```
/*
 * Prepare next lot of parameters for ICL,
 * and see to any current events in the synthesis list.
 * Returns 0 if the end of the synthesis list is encountered, otherwise returns 1.
 */
iclparm()
{
    register int i, t, x;
    int iclp[NLOG];
    int e, j, y;

    pd[ICLTYP].clock++;
/* update current logical parameter values */
    for (i = 0; i < NLOG; i++) {
        pd[ICLTYP].values[i] =+ pd[ICLTYP].rates[0][i];
        if (--pd[ICLTYP].rates[1][i] == 0) pd[ICLTYP].rates[0][i] = 0;
    }

/* dequeue any events, and deal with them */
    for (ever) {
        if (pd[ICLTYP].parmp == sd[ICLTYP].lastp) {
            pd[ICLTYP].error[2]++;
            break;
        }
        if ((x = pd[ICLTYP].parmp->content[1] - pd[ICLTYP].clock) > 0) break;
        if ((e = pd[ICLTYP].parmp->content[0]) == EOUTYP) {
            freeit(pd[ICLTYP].parmp);
```

```
            return(0);
        }
        if (x < 0) {
            pd[ICLTYP].error[1]++;
            pd[ICLTYP].clock = pd[ICLTYP].parmp->content[1];
        }
        else if ((t = pd[ICLTYP].parmp->content[3]) <= 0) {
            pd[ICLTYP].error[3]++;
            pd[ICLTYP].parmp->content[3] = 1; /* set to good value */
        }
        else if (e == ASPTYP) pd[ICLTYP].asp = t;
        else {
            i = pd[ICLTYP].parmp->content[2];
            x = pd[ICLTYP].parmp->content[4];
            if (e == TARGTYP) {
                pd[ICLTYP].rates[0][i] = (x - pd[ICLTYP].values[i] + (t >> 1))/t;
                pd[ICLTYP].rates[1][i] = t;
            }
            else {
                pd[ICLTYP].incrs[0][i] = x;
                pd[ICLTYP].incrs[1][i] = t;
            }
        }
        pd[ICLTYP].parmp = freeit(pd[ICLTYP].parmp);
    }

/* compute current parameter values */
    for (i = 0; i < NLOG; i++) {
        iclp[i] = (pd[ICLTYP].values[i] + pd[ICLTYP].incrs[0][i]) >> 5;
        if (-- pd[ICLTYP].incrs[1][i] == 0) pd[ICLTYP].incrs[0][i] = 0;
    }

/* re-organize parameter values for ICL */
    pd[ICLTYP].parms[6] = iclp[0];   /* A1 */
    pd[ICLTYP].parms[0] = iclp[6];   /* F0 */
    pd[ICLTYP].parms[1] = iclp[2];   /* F1 */
    pd[ICLTYP].parms[2] = iclp[3];   /* F2 */
    i = ((iclp[3] + (iclp[2]>>1))/iclp[2]) << 2;
    for (j = 0; i =>> 1; j++);
    pd[ICLTYP].parms[3] = j;   /* A2 */
    if (pd[ICLTYP].asp) {
        pd[ICLTYP].asp--;
        pd[ICLTYP].parms[4] = 3;   /* sound type */
        pd[ICLTYP].parms[5] = iclfric[0];   /* fricative amplitude */
        pd[ICLTYP].parms[7] = 3;   /* D-S */
    }
    else if (iclp[1] == 0) {
        pd[ICLTYP].parms[4] = iclp[4];   /* F3 */
        i = ((iclp[4] + (iclp[3]>>1))/iclp[3]);
        for ( ; i =>> 1; j++);
        pd[ICLTYP].parms[5] = j;   /* A3 */
        pd[ICLTYP].parms[7] = 0;   /* D-S */
    }
    else {
        if (iclp[0]) {
            i = (iclp[0] << iclfric[1])/iclp[1];
            for (j = 0; i =>> 1; j++);
            pd[ICLTYP].parms[5] = j;   /* fricative amplitude */
            pd[ICLTYP].parms[7] = 1;   /* D-S */
        }
        else {
            pd[ICLTYP].parms[5] = iclfric[0];   /* fricative amplitude */
            pd[ICLTYP].parms[6] = iclp[1];   /* A1 */
            pd[ICLTYP].parms[7] = 3;   /* D-S */
        }
        x = iclp[5]<<4;
        j = 10000;
        for (i = 0; i < 5; i++) {
            y = iclmisc[i] - x;
            if (y < 0) y = -y;
            if (y < j) {
                pd[ICLTYP].parms[4] = i;   /* sound type */
                j = y;
            }
        }
    }
    return(1);
}


/*
 * Scale logical parameter number i, value x, into a 9-bit quantity.
 */
int iclscale(i, x)
int i, x;
{
    register int y, z;

    switch (i) {
    case 0:
    case 1:
        return(x<<3);
    case 2:
    case 3:
    case 4:
        return(scalediv(x, 8, 10000));
    case 5:
        return(x>>4);
    case 6:
        return((10000 + (x>>1))/x);
    }
}



/*
 * Limit logical parameter number i, value x, into appropriate range.
 */
icllimit(i, x)
int i, x;
{
    if (x < 0) {
```

```
        pd[ICLTYP].error[4]++;
        return(0);
    }
    switch (i) {
    case 0:
    case 1:
    case 5:
        if (x > 0777) {
            pd[ICLTYP].error[4]++;
            return(0777);
        }
        return(x);
    case 2:
    case 3:
    case 4:
    case 6:
        if (x > 0377) {
            pd[ICLTYP].error[4]++;
            return(0377);
        }
        return(x);
    }
}
```

```
/*
 * COMPUTALKER calibration data
 *
 *      The array comcal[] gives the minimum value and the
 *      maximum value, for each parameter.
 */
int comcal[7][2] {
       0,  256,     /*  av  */
       0,  384,     /*  as  */
    1450,  170,     /*  f1  */
    4400,  520,     /*  f2  */
    5500, 1700,     /*  f3  */
   14000, 1700,     /*  fh  */
      74,  470};    /*  f0  */

/*
 * COMPUTALKER handler.
 * Enter on COMPUTALKER interrupt.
 */
int compending 0;    /* incremented whenever an interrupt occurs during output
                        to COMPUTALKER or during comparm; in which case the interrupt is
                        held pending completion of comparm    */
computalker()
{
    register int i, j;

/* check for illegal state */
    if ((i=pd[COMTYP].state) <= 0 || i > 4) abortit("com: illegal COM interrupt");
```

```
/* ignore the very first interrupt  */
    if(pd[COMTYP].clock <-1) {
        pd[COMTYP].clock++;
        return;
    }
/* check for re-entrancy */
    if (i == 2 || i == 4) if (compending++) pd[COMTYP].error[0]++;  else;
    else {
        pd[COMTYP].state++;
        compending++;
        while (compending) {
            compending = 0;
            mtps(PLOW);  /* make interruptable */
/* output pd[COMTYP].parms[] to COMPUTALKER */
            j = 0;
            for (i=0; i<NPHYS; i++) {
                COMADDR->writebuf = pd[COMTYP].parms[i] | j;
                j =+ (1<<6);
            }
/* get next lot of parameters ready */
            i = comparm();
            mtps(PHIGH);
        }
        if (i) {
/* exit to seg if necessary */
            if (pd[COMTYP].state == 2 && sd[COMTYP].sclock-pd[COMTYP].clock < ADVANCLOCK)
                pd[COMTYP].state++;
                mtps(PLOW);
                seg(COMTYP);
                mtps(PHIGH);
                pd[COMTYP].state--;
            }
            pd[COMTYP].state--;
        }
        else {
            COMADDR->csr =& ~INTRUPTA;
            COMADDR->writebuf = 15*(1<<6) + 0;  /* shut up COMPUTALKER */
            pd[COMTYP].state = 0;
            compending=0;
        }
    }
}
```

```
/*
 * Prepare next lot of parameters for COMPUTALKER,
 * and see to any current events in the synthesis list.
 * Returns 0 if the end of the synthesis list is encountered; otherwise returns 1.
 */
comparm()
{
    register int i, t, x;
    int e;

    pd[COMTYP].clock++;
/* update current logical parameter values */
```

```
    for (i = 0; i < NLOG; i++) {
        pd[COMTYP].values[i] =+ pd[COMTYP].rates[0][i];
        if (--pd[COMTYP].rates[1][i] == 0) pd[COMTYP].rates[0][i] = 0;
    }

/* dequeue any events, and deal with them */
    for (ever) {
        if (pd[COMTYP].parmp == sd[COMTYP].lastp) {
            pd[COMTYP].error[2]++;
            break;
        }
        if ((x = pd[COMTYP].parmp->content[1] - pd[COMTYP].clock) > 0) break;
        if ((e = pd[COMTYP].parmp->content[0]) == EOUTYP) {
            freeit(pd[COMTYP].parmp);
            return(0);
        }
        if (x < 0) {
            pd[COMTYP].error[1]++;
            pd[COMTYP].clock = pd[COMTYP].parmp->content[1];
        }
        if ((t = pd[COMTYP].parmp->content[3]) <= 0) {
            pd[COMTYP].error[3]++;
            pd[COMTYP].parmp->content[3] = 1; /* set it to a safe value */
        }
        else if (e == ASPTYP) pd[COMTYP].asp = t;
        else {
            i = pd[COMTYP].parmp->content[2];
            x = pd[COMTYP].parmp->content[4];
            if (e == TARGTYP) {
                pd[COMTYP].rates[0][i] = (x - pd[COMTYP].values[i] + (t >> 1))/t;
                pd[COMTYP].rates[1][i] = t;
            }
            else {
                pd[COMTYP].incrs[0][i] = x;
                pd[COMTYP].incrs[1][i] = t;
            }
        }
        pd[COMTYP].parmp = freeit(pd[COMTYP].parmp);
    }

/* compute current parameter values */
    for (i = 0; i < NLOG; i++) {
        pd[COMTYP].parms[i] = comlimit(pd[COMTYP].values[i]
                                        + pd[COMTYP].incrs[0][i] + 32 >> 6);
        if (--pd[COMTYP].incrs[1][i] == 0) pd[COMTYP].incrs[0][i] = 0;
    }

/* shuffle parameter values for COMPUTALKER */
    x = pd[COMTYP].parms[6];
    pd[COMTYP].parms[6] = pd[COMTYP].parms[1];
    pd[COMTYP].parms[1] = x;
    pd[COMTYP].parms[7] = pd[COMTYP].parms[5];
    if (pd[COMTYP].asp) {
        pd[COMTYP].asp--;
        pd[COMTYP].parms[5] = comlimit(pd[COMTYP].parms[0]);
        pd[COMTYP].parms[0] = 0;
    }
    else pd[COMTYP].parms[5] = 0;

    return(1);
}


/*
 * Scale parameter number i, value x, into a 8-bit quantity
 * according to comcal[][].
 */
int comscale(i, x)
int i, x;
{
    if (2 <= i && i <= 6)  return(logscale(x, comcal[i][0], comcal[i][1], 8, COMTYP));

    if (comcal[i][0] == 0 && comcal[i][1] == 256) return(x<<2);

    return(comlimit(scalediv(x-comcal[i][0], 8, comcal[i][1]-comcal[i][0])));
}


/*
 * Limit argument to range [0,255].
 */
int comlimit(x)
int x;
{
    if (x < 0) {
        pd[COMTYP].error[4]++;
        return(0);
    }
    if (x > 255) {
        pd[COMTYP].error[4]++;
        return(255);
    }
    return(x);
}



#define PARALLEL 0
#define SERIAL 1
#define DHADDR  0167770
#define CONSADD 0177560
#define RQA     0200
#define RQB     0100000
#define RQAEN   0100
#define RQBEN   0040
#define READY   0200
#define CSR0    01
#define CSR1    02

struct drv11 {
        int parcsr;
```

```
        int parobuf;
        int paribuf;
};
struct dlv11 {
        int sericsr;
        int seribuf;
        int serocsr;
        int serobuf;
};

char    *wrong[]{
        "bus error",
        "illegal instruction",
        "breakpoint",
        "IOT trap",
        "powerfail",
        "EMT instruction",
        "trap instruction",
        "console input",
        "console output",
        "DR-(70) rqa",
        "DR-(70) rqb",
        "DR-(60) rqa",
        "DR-(60) rqb",
        "DR-(50) rqa",
        "DR-(50) rqb",
};


/*      This routine is called from the interrupt level on all TRAPS
        r0 and r1 are saved (cos C doesn't) so that a return could be
        made to the trapped program if that is sensible.
        nps  - is the new status (from the vector)
        type - is the type of trap that occurred (from 4 lsbits of nps)
*/

trap(type,r1,nps,r0,pc,ps)
int     type,r1,nps,r0,pc,ps;
{
        abortit(wrong[type]);
}


char jgetchar(channel)
int channel;
{
register int *addr;
char c;
        if(channel == PARALLEL) return(getdr(0));
        addr = CONSADD;
        while ((addr->sericsr & READY) == 0);
        c = addr->seribuf &0177;
        if((c >= 'A') && (c <= 'Z')) c =& 0137;
        jputchar(c,SERIAL);
        return(c);
}

jputchar(c,channel)
char c;
int channel;
{
register int *addr;
int     i;
        if(channel == PARALLEL){
                putdr(c,0);
                return;
                }
        addr = CONSADD;
        while ((addr->serocsr & READY) == 0);
        addr->serobuf = c & 0377;
}



char getdr(dev)
int dev;
{
register int *addr;
        addr = DRADDR-(dev<<3);
        while ((addr->parcsr & RQB) == 0);
        return(addr->paribuf & 0377);
}

putdr(w,dev)
int dev;
char w;
{
register int *addr;
        addr = DRADDR-(dev<<3);
        while ((addr->parcsr & RQA) == 0);
        addr->parobuf = w;
}

setprior(prior)
int prior;
{
        int oldprior;
        char psw;

        psw = mfps();
        oldprior = psw & 0200;
        psw =& 0177;
        if (prior) psw =| 0200;
        mtps(psw);
        return(oldprior);
}
```